SIGBIM

Sistema de Gestión de Bodega de Insumos Médicos

Informe Técnico Integrador — SIGBIM

Documentación completa del proyecto: análisis y diseño (Hito 1), construcción, pruebas y puesta en marcha (Hito 2) y versión final multiplataforma (macOS)

Proyecto	SIGBIM — Sistema de Gestión de Bodega de Insumos Médicos
Asignatura	Desarrollo e Ingeniería de Software (PRO205)
Institución	AIEP — Universidad Andrés Bello
Estudiante	Jonathan Felipe Urbina Monasterio
Correo	JONATHAN.URBINA@correoaiep.cl
Modalidad	Trabajo individual
Documento	Documento único consolidado (documentos 01 a 10 de 10)
Repositorio	PRO205-JONATHAN.URBINA (estructura /docs, /database, /src, /scripts)
Fecha de emisión	18 de octubre de 2026

Contenido del documento único

Parte 1 — Informe Técnico del Proyecto

- Resumen ejecutivo
- Descripción y delimitación de la problemática
- Objetivos
- Actores del sistema
- Alcance funcional (módulos del sistema)
- Aplicación de la norma ISO/IEC 27001:2022
- Modelo de seguridad y trazabilidad diseñado
- Normativa y estándares de referencia
- Tecnologías y arquitectura (resumen)
- Metodología, cronograma y control de versiones
- Riesgos y mitigaciones
- Criterios de aceptación del proyecto (Definition of Done)
- Conclusiones del análisis
- Anexos e índice de documentos

Parte 2 — Especificación de Requerimientos

- Convenciones de especificación
- Requisitos funcionales
- Requisitos no funcionales
- Matriz de trazabilidad: requisitos ↔ controles ISO/IEC 27001:2022 ↔ evidencia

Parte 3 — Modelo de Casos de Uso (UML)

- Actores
- Diagrama de casos de uso (general)
- Especificación textual de casos de uso
- Matriz actor ↔ caso de uso

Relaciones <<include>> y <<extend>> justificadas

Parte 4 — Diagrama de Clases (UML)

Diagrama de clases del dominio

Descripción de las clases del dominio

Reglas de negocio expresadas en los métodos del dominio

Diagrama de clases de la arquitectura (mapeo a las capas)

Correspondencia clase de dominio ↔ tabla del modelo relacional

Parte 5 — Modelo Relacional y Normalización

Modelo relacional (notación)

Normalización (1FN → 3FN)

Integridad referencial y restricciones de integridad

Vista de apoyo y decisiones de modelado

Parte 6 — Arquitectura de la Aplicación

Vista general

Responsabilidades por capa

Flujo de datos extremo a extremo

Validaciones y manejo de errores por capa

Estructura de la solución y patrones aplicados

Trazabilidad arquitectura ↔ requisitos ↔ rúbrica

Parte 7 — Plan de Pruebas y Evidencias

Estrategia de pruebas

Casos de prueba

Matriz de cobertura requisito ↔ caso de prueba

Evidencias

Trazabilidad con los controles ISO/IEC 27001:2022

Parte 8 — Manual de Ejecución en Windows

Requisitos del equipo

Instalación del entorno (una sola vez)

Creación de la base de datos

Configuración de la conexión (fuera del código fuente)

Compilación y ejecución

Respaldo de la base de datos

Diferencias entre macOS (desarrollo) y Windows (entrega)

Solución de problemas frecuentes

Resumen del proceso macOS → Windows

Parte 9 — Guía Paso a Paso y Justificación

Mapa del repositorio (dónde está cada cosa)

Paso a paso de construcción (en el orden en que se realizó)

Justificación de las decisiones técnicas

Cómo responde el proyecto a cada criterio de la rúbrica

Guion de demostración (secuencia sugerida para la entrega)

Estado de la entrega y pendientes por entorno

Conclusión

Parte 10 — Versión Final Multiplataforma (macOS)

Por qué existe una versión para macOS

Alcance del producto mínimo viable

Arquitectura y menú de la versión final

Funcionalidades agregadas en la versión final

Seguridad aplicada (ISO/IEC 27001:2022)

Pruebas y evidencias

Ejecución en macOS

Correspondencia con la rúbrica

Mejoras futuras

PARTE 1 DE 10

Informe Técnico del Proyecto

Problemática, objetivos, actores y controles ISO/IEC 27001:2022

Documento 01 del repositorio (docs/01_Informe_Tecnico.md)

Ficha del documento	
Proyecto	Sistema de Gestión de Bodega de Insumos Médicos (SIGBIM)
Asignatura	Desarrollo e Ingeniería de Software (PRO205)
Institución	AIEP — Universidad Andrés Bello
Estudiante	Jonathan Felipe Urbina Monasterio — `JONATHAN.URBINA@correoaiep.cl`
Modalidad	Trabajo individual
Repositorio	`PRO205-JONATHAN.URBINA` (estructura `/docs`, `/database`, `/src`, `/scripts`)
Stack	C# .NET 8 (Windows Forms) · SQL Server (T-SQL) · arquitectura en 4 capas · Git/GitHub
Fecha de emisión	septiembre de 2026

1. Resumen ejecutivo

El presente informe documenta el análisis, la especificación de requisitos y el diseño del **SIGBIM**, una aplicación de escritorio para Windows Forms que reemplaza el control manual (planillas Excel compartidas) de la bodega de insumos médicos de un centro asistencial de atención ambulatoria, por un **mantenedor de entidad** centralizado sobre SQL Server, con **gestión de seguridad** (autenticación, roles y permisos) y **trazabilidad completa** (auditoría en base de datos vía triggers, registro de eventos de seguridad y reportes en PDF).

La solución responde a tres necesidades reales del negocio: (i) **integridad y disponibilidad** del dato de inventario crítico, (ii) **confidencialidad y control de acceso** sobre información institucional sensible (ISO/IEC 27001:2022, controles A.5.15, A.5.17, A.8.2, A.8.3, A.8.5, A.8.15, A.8.16) y (iii) **evidencia auditable** de quién hizo qué, cuándo y sobre qué registro (A.8.15 Registro de eventos, A.8.16 Actividades de supervisión).

El diseño se entrega en los documentos `02` a `06` de este repositorio y su implementación en `/database` (script T-SQL con procedimientos almacenados y triggers) y `/src` (solución .NET en capas UI, BLL, DAL y Entidades).

2. Descripción y delimitación de la problemática

2.1 Contexto real y caso aplicable

En el centro asistencial donde se aplica este caso (atención ambulatoria de baja complejidad, con una bodega de insumos clínicos), el inventario de **insumos médicos** —guantes de procedimiento, jeringas, gasas, suturas, alcohol gel, apósitos, catéteres, entre otros— se administra hoy con **planillas Excel compartidas por correo** y con registros en papel para los movimientos diarios (entregas a boxes de atención, reposiciones y ajustes por vencimiento). No existe un sistema informático único ni historial formal de cambios.

2.2 Situación problemática identificada

#	Problema observado	Consecuencia operativa / de seguridad
P1	Múltiples versiones de la planilla maestra (Excel) sin control de versiones	Datos divergentes; decisiones de compra sobre información errónea
P2	Sin autenticación ni perfiles: cualquier equipo de la red puede abrir y editar el archivo	Violación de control de acceso (ISO 27001 A.5.15/A.8.3); alteración no autorizada de datos
P3	Sin trazabilidad: no se sabe quién modificó precios, stock o datos de proveedores	Imposible investigar diferencias de inventario ni responder a una auditoría (A.8.15/A.8.16)
P4	Sin alertas de stock crítico ni de vencimientos	Desabastecimiento de insumos clínicos y pérdidas por vencimiento
P5	Sin validación de datos (SKU duplicados, precios negativos, stock inconsistente)	Registros duplicados e inconsistentes; reportes poco confiables
P6	Sin respaldo ni esquema de retención de la información	Riesgo de pérdida total del histórico de inventario
P7	Sin evidencia documental del control de inventario	Hallazgos en auditorías internas / fiscalizaciones sanitarias

2.3 Delimitación del alcance (qué resuelve el SIGBIM)

Dentro del alcance del sistema (implementación del Hito 2):

1. **Mantenedor de la entidad principal: Insumo médico (`Producto`)**, con datos maestros asociados (`Categoria`, `Proveedor`, `UnidadMedida`) y CRUD completo (alta, consulta con filtros, actualización y baja lógica) ejecutado **exclusivamente a través de procedimientos almacenados**.
2. **Ingesta y egreso de stock (`MovimientoStock`)**: entradas por compra/reposición, salidas a boxes de atención y ajustes por merma o vencimiento, con validación de que el stock nunca resulte negativo.
3. **Gestión de seguridad**: autenticación local con contraseñas derivadas (PBKDF2-SHA256) y sal, bloqueo por intentos fallidos, administración de usuarios, roles y permisos por funcionalidad, y sesión de aplicación con cierre controlado.
4. **Trazabilidad y auditoría**: triggers `AFTER INSERT/UPDATE/DELETE` sobre las tablas maestras que escriben en `Auditoria` con valores anteriores y nuevos en JSON, más registro de eventos de seguridad (inicio de sesión exitoso/fallido, acceso denegado, generación de reportes).
5. **Reportes en PDF**: informe del **log de auditoría** (obligatorio) filtrable por rango de fechas, usuario, tabla y operación, y reporte complementario de **stock crítico**.

Fuera del alcance (declarado explícitamente):

- Facturación, compras a proveedores y cuentas por pagar (solo se registra el documento de referencia del movimiento, no su ciclo contable).
- Trazabilidad unitaria por lote/serie y control de temperatura de cadena de frío.
- Integración con sistemas institucionales (ERP, HIS) y despliegue en red multiusuario; el Hito 2 se ejecuta en un entorno local (equipo + SQL Server local/LocalDB).
- Autenticación contra directorio corporativo (LDAP/Active Directory) y firma electrónica; se documentan como evolución futura.

2.4 Beneficio esperado

Un único punto de verdad para el inventario de insumos, con **control de acceso por rol**, **historial inmutable de cambios** y **evidencia en PDF** para auditoría interna, reduciendo quiebres de stock y pérdidas por vencimiento, y habilitando la rendición de cuentas sobre el uso de insumos clínicos.

---

3. Objetivos

3.1 Objetivo general

Diseñar, documentar e implementar una aplicación de escritorio (C# .NET 8 / Windows Forms) con arquitectura en cuatro capas que administre el mantenedor de insumos médicos sobre una base de datos relacional normalizada en SQL Server, garantizando control de acceso por roles, integridad referencial y trazabilidad auditable de todas las operaciones, con generación de reportes en PDF del registro de auditoría.

3.2 Objetivos específicos

1. Definir y documentar la problemática, el alcance y los actores del proceso de control de bodega de insumos.
2. Especificar los requisitos funcionales y no funcionales aplicando la **tríada** (rol + necesidad + criterio de aceptación/seguridad) e integrar los controles de seguridad de **ISO/IEC 27001:2022**.
3. Modelar las interacciones actor–sistema mediante **diagramas UML de casos de uso** y la estructura del dominio mediante un **diagrama de clases** con visibilidad, atributos, métodos y relaciones.
4. Transformar el modelo de clases en un **modelo relacional normalizado (1FN, 2FN, 3FN)** con integridad referencial, restricciones `CHECK` / `UNIQUE` y control de concurrencia optimista.
5. Implementar en T-SQL el esquema, la totalidad de los **procedimientos almacenados** para las operaciones de la aplicación y los **triggers de auditoría** que alimentan la tabla de log.
6. Implementar la aplicación en cuatro capas (**UI, BLL, DAL, Entidades**) en Visual Studio Code, sin sentencias SQL directas en la interfaz gráfica: todo el acceso a datos se realiza invocando procedimientos almacenados.
7. Incorporar validaciones en UI y en BLL más manejo de excepciones de base de datos con mensajes funcionales.
8. Generar reportes **PDF** del registro de auditoría (log) y de stock crítico, y versionar todo el avance en GitHub con commits incrementales y descriptivos.

---

4. Actores del sistema

Actor	Descripción	Interacción principal
Administrador del sistema	Encargado de TI / jefatura de la bodega. Responsable de la seguridad de la información	Administra usuarios, roles y permisos; consulta el log de auditoría; genera reportes PDF
Bodeguero / Técnico de bodega	Usuario operativo que mantiene el inventario de insumos	CRUD de insumos, categorías y proveedores; registra movimientos de stock
Encargado de bodega (jefatura)	Responsable del abastecimiento clínico	Consulta existencias críticas y vencimientos; genera reporte PDF de stock crítico
Auditor interno / Fiscalizador sanitario	Revisa el cumplimiento del control interno	Consulta y genera el reporte PDF del log de auditoría (solo lectura)
Base de datos (SQL Server)	Actor de sistema	Ejecuta los procedimientos almacenados, aplica las restricciones de integridad y dispara los triggers de auditoría

---

5. Alcance funcional (módulos del sistema)

Módulo	Formulario(s) WinForms	Funcionalidad	SPs asociados
Autenticación	`FrmLogin`	Inicio de sesión, bloqueo por intentos, cambio de contraseña	`sp_Usuario_ObtenerPorNombre`, `sp_Usuario_RegistrarIntentoFallido`, `sp_Usuario_RegistrarAcceso`, `sp_Usuario_CambiarPassword`
Menú principal	`FrmPrincipal`	Navegación según permisos del rol y barra de estado con usuario de sesión	— (consume BLL)
Categorías	`FrmCategorias`	CRUD de categorías de insumos	`sp_Categoria_*`
Proveedores	`FrmProveedores`	CRUD de proveedores (RUT, contacto, ciudad)	`sp_Proveedor_*`

Módulo	Formulario(s) WinForms	Funcionalidad	SPs asociados
Insumos (entidad principal)	`FrmProductos`	CRUD con SKU único, precios, stock mínimo, vencimiento, baja lógica y filtros	`sp_Producto_*`
Movimientos de stock	`FrmMovimientos`	Entradas, salidas y ajustes; consulta histórica por insumo y rango de fechas	`sp_Movimiento_*`
Seguridad	`FrmUsuarios`	CRUD de usuarios, asignación de rol, activación/desactivación y reseteo de contraseña	`sp_Usuario_*`, `sp_Rol_*`, `sp_Permiso_*`
Trazabilidad	`FrmAuditoria`	Consulta filtrada del log de auditoría y exportación a PDF	`sp_Log_Listar`, `sp_Reporte_Auditoria`
Reportes	`FrmAuditoria`, `FrmProductos`	Generación de PDF de auditoría y de stock crítico	`sp_Reporte_StockCritico`

---

6. Aplicación de la norma ISO/IEC 27001:2022

Los requisitos del SIGBIM se especifican en `docs/02_Requerimientos_RF_RNF_ISO27001.md`. Cada requisito funcional se redacta con la **tríada obligatoria**: *Rol/Usuario + Necesidad/Acción + Criterio de aceptación/seguridad*, y se vincula a uno o más controles del Anexo A de ISO/IEC 27001:2022.

6.1 Controles del Anexo A aplicados al proyecto

Control ISO 27001:2022	Denominación	Aplicación concreta en el SIGBIM
A.5.15	Control de acceso	Autorización por rol/permiso validada en la BLL antes de invocar cada SP (`SeguridadService.TienePermiso`)
A.5.16	Gestión de identidades	Ciclo de vida del usuario en la tabla `Usuario`: alta, modificación, activación/desactivación (baja lógica)
A.5.17	Información de autenticación	Contraseñas derivadas con PBKDF2-SHA256, 100.000 iteraciones y sal aleatoria de 16 bytes; nunca se registran en el log
A.5.18	Derechos de acceso	Permisos granulares en `Permiso` / `RolPermiso`; solo el Administrador gestiona usuarios y roles
A.8.2	Privilegios de acceso	Rol `Consulta` de solo lectura; el Bodeguero no administra usuarios ni elimina movimientos
A.8.3	Restricción de acceso a la información	Bloqueo en la UI de las funcionalidades no asignadas al rol y filtrado de registros activos
A.8.5	Autenticación segura	Bloqueo temporal tras 5 intentos fallidos (`IntentosFallidos`, `BloqueadoHasta`) y política de robustez de contraseña
A.8.9	Gestión de configuraciones	Cadena de conexión fuera del código fuente (`appsettings.json` / variable de entorno)
A.8.11	Enmascaramiento de datos	El trigger de `Usuario` registra hash y sal como `***` en el log (no se exportan a PDF)
A.8.12	Prevención de fuga de información	El reporte PDF no expone credenciales; los archivos de configuración quedan excluidos por `.gitignore`
A.8.13	Copias de seguridad	Procedimiento de respaldo/restauración documentado en `scripts/` y semilla reproducible
A.8.15	Registro de eventos (logging)	Triggers `AFTER INSERT/UPDATE/DELETE` sobre las tablas maestras hacia la tabla `Auditoria`, con valores previos y nuevos
A.8.16	Actividades de supervisión	Consulta filtrada del log y reporte PDF para revisión periódica de jefatura/auditoría
A.8.24	Uso de criptografía	Derivación de claves PBKDF2-SHA256 con sal; `TrustServerCertificate` habilitado solo en el entorno local de desarrollo
A.8.28	Codificación segura	Consultas 100 % parametrizadas ejecutadas con `CommandType.StoredProcedure`; `TRY/CATCH` y `THROW` en los SPs; sin concatenación de SQL (mitiga inyección SQL)

6.2 Trazabilidad requisito ↔ control ↔ evidencia

La matriz completa requisito ↔ control ↔ criterio de aceptación ↔ evidencia se encuentra en `docs/02_Requerimientos_RF_RNF_ISO27001.md` (secciones 2 y 3), y la estrategia de pruebas con evidencias en `docs/07_Plan_de_Pruebas_y_Evidencias.md`.

---

7. Modelo de seguridad y trazabilidad diseñado

7.1 Autenticación (A.5.17, A.8.5)

- Usuario + contraseña local contra la tabla `Usuario` (sin exponer si el usuario existe: mensaje genérico).
- Derivación de clave: **PBKDF2-SHA256**, 100.000 iteraciones, sal aleatoria de 16 bytes por usuario (`Rfc2898DeriveBytes` en C#); en la base solo se almacena el hash y la sal en Base64.
- Política de contraseña: mínimo 10 caracteres, mayúscula, minúscula, dígito y símbolo, distinta al nombre de usuario; `DebeCambiarPassword` obliga el cambio en el primer acceso.
- **Bloqueo**: a los 5 intentos fallidos consecutivos se bloquea la cuenta 15 minutos (`IntentosFallidos`, `BloqueadoHasta`), lo que se registra como evento `LOGIN_FALLIDO`.

7.2 Autorización (A.5.15, A.5.18, A.8.2, A.8.3)

- Modelo **RBAC**: `Rol` → `RolPermiso` → `Permiso` (códigos como `PRODUCTO_CREAR`, `LOG_VER`, `REPORTE_PDF`).
- La BLL resuelve los permisos del usuario autenticado y la UI habilita/deshabilita funcionalidades; toda operación sensible vuelve a validar el permiso en la capa de negocio antes de invocar el SP.
- Roles semilla: **Administrador** (todos los permisos), **Bodeguero** (mantenedores y movimientos; sin gestión de usuarios), **Consulta** (solo lectura y reportes).

7.3 Auditoría y trazabilidad (A.8.15, A.8.16)

- Triggers `AFTER INSERT/UPDATE/DELETE` en `Producto`, `Categoria`, `Proveedor`, `UnidadMedida`, `Usuario` y `MovimientoStock`: registran tabla, operación, id del registro, usuario de aplicación (`SESSION_CONTEXT('UsuarioApp')`), usuario de base (`SUSER_SNAME()`), equipo (`HOST_NAME()`), fecha, y los valores anteriores/nuevos en JSON (`FOR JSON PATH`).
- En la tabla `Usuario` el trigger **enmascara** hash y sal (`***`) — A.8.11.
- Eventos de aplicación (login exitoso/fallido, acceso denegado, generación de reporte PDF) se registran con `sp_Log_Registrar`, garantizando trazabilidad del uso del propio sistema.
- Retención definida: **12 meses** de log en línea; el borrado lógico de maestros mantiene el histórico íntegro.

7.4 Datos, respaldo y criptografía (A.8.13, A.8.24, A.8.12)

- La base no almacena datos clínicos de pacientes (minimización de datos); la información es institucional.
- Cadena de conexión fuera del código fuente (`appsettings.json` ignorado por git, o variable `SIGBIM_CONN`) con autenticación integrada de Windows en el entorno del centro asistencial.
- Respaldo completo diario y respaldo de log; procedimiento documentado en `scripts/respaldo_bd.sql` + `scripts/README.md`.

---

8. Normativa y estándares de referencia

Referencia	Uso en el proyecto
IEEE 830 / ISO/IEC/IEEE 29148	Estructura de la especificación de requisitos (identificación, prioridad, criterio de aceptación verificable)
ISO/IEC 27001:2022 (Anexo A)	Controles de seguridad aplicados y matriz de cumplimiento (sección 6.1)
ISO/IEC 27002:2022	Guía de implementación de los controles A.5.x y A.8.x
OWASP Top 10 (2021/2025)	A03 Inyección (consultas parametrizadas + SPs), A07 Fallas de identificación (bloqueo/PBKDF2), A09 Fallas de registro y monitoreo (trigger de auditoría)
Ley 19.628 / 21.719 (Chile)	Protección de datos personales: minimización, no registro de credenciales, acceso restringido
UML 2.5	Casos de uso y diagrama de clases
Normas de normalización (Codd)	1FN, 2FN y 3FN del modelo relacional

---

9. Tecnologías y arquitectura (resumen)

Componente	Tecnología	Justificación
Interfaz de usuario	C# .NET 8 · Windows Forms (formularios construidos en código, compatibles con VS Code)	Requisito del encargo; sin dependencia del diseñador gráfico
Lógica de negocio	Biblioteca de clases `PRO205.BLL`	Centraliza validaciones, permisos y reglas (no puede reutilizarse el estado de stock desde la UI)
Acceso a datos	`PRO205.DAL` con `Microsoft.Data.SqlClient`, solo `CommandType.StoredProcedure`	Mitiga inyección SQL y concentra el manejo de errores de base de datos
Entidades	`PRO205.Entidades` (POCO)	Transporte de datos entre capas, sin lógica
Reportes	`PRO205.Reportes` con QuestPDF (cross-platform)	Generación programática de PDF sin dependencias de Office
Base de datos	SQL Server (T-SQL), SPs, triggers, restricciones e índices	Motor exigido; integridad y trazabilidad en el propio motor
Control de versiones	Git + GitHub (`/docs`, `/database`, `/src`)	Entregable obligatorio con historial incremental

El detalle estructural (responsabilidades por capa, flujo de datos y reglas de dependencia) está en `docs/06_Arquitectura_4_Capas.md`.

---

10. Metodología, cronograma y control de versiones

10.1 Enfoque de trabajo

Desarrollo incremental por hitos con verificación en cada entrega; el repositorio GitHub es la fuente única de verdad y el historial de commits evidencia la evolución del trabajo:

Fase	Actividad	Artefactos	Estado
F1	Análisis, problemática, requisitos con tríada e ISO 27001	`docs/01`, `docs/02`	Entregado
F2	Modelado UML, modelo relacional normalizado y arquitectura	`docs/03` a `docs/06`	Entregado
F3	Esquema T-SQL, procedimientos almacenados, triggers de auditoría y semilla	`database/01` a `database/04`	Entregado
F4	Solución .NET en 4 capas y módulo de reportes PDF	`src/`	Entregado
F5	Pruebas, evidencias y manual de ejecución en Windows	`docs/07`, `docs/08`, `docs/evidencias`	Entregado

10.2 Estrategia de commits (evidencia de avance incremental)

- Un commit por unidad lógica terminada, por ejemplo `docs: especificación de requisitos RF/RNF con controles ISO 27001`, `database: script DDL normalizado con integridad referencial`, `src(bll): validaciones y reglas de stock`, `src(ui): formulario del mantenedor de insumos`.
- Mensajes descriptivos en formato `área: descripción imperativa` (estilo Conventional Commits), sin commits masivos ni genéricos.
- Estructura versionada según lo exigido: `/docs`, `/database`, `/src`.

10.3 Entorno de desarrollo y entrega

El análisis, la codificación y la verificación de compilación se realizaron en macOS (compilación de todas las capas, incluida la UI, mediante `EnableWindowsTargeting`, y ejecución del proyecto `PRO205.SmokeTest` para validar DAL, BLL y la generación de PDF). La ejecución final del mantenedor WinForms y de los scripts T-SQL se realiza en **Windows** con SQL Server local, conforme al entorno exigido por la asignatura: el procedimiento está documentado en `docs/08_Manual_Ejecucion_Windows.md`.

---

11. Riesgos y mitigaciones

Riesgo	Probabilidad	Impacto	Mitigación implementada
Divergencia entre el entorno macOS (desarrollo) y Windows (entrega)	Media	Alto	UI construida en código (sin diseñador gráfico), compilación cruzada con `EnableWindowsTargeting` y cadena de conexión externalizada: el port requiere solo ajustar `appsettings.json`
Errores de datos que afecten el inventario (stock negativo, duplicados)	Media	Alto	Restricciones `CHECK`, `UNIQUE`, `FK`, validaciones en BLL y reglas dentro de los SPs con `THROW`
Acceso no autorizado o abuso de privilegios	Baja	Alto	RBAC, contraseñas derivadas, bloqueo por intentos y registro de eventos de seguridad
Pérdida de información	Baja	Alto	Respaldo documentado, semilla reproducible y baja lógica en lugar de borrado físico
Pérdida de trazabilidad (cambios sin registro)	Baja	Medio	Triggers en el motor de base de datos (no dependen del código de la aplicación)
Cambios concurrentes sobrescritos	Media	Medio	Concurrencia optimista con `RowVersion` validado dentro del SP
Entrega fuera de plazo del hito	Media	Medio	Alcance delimitado (sección 2.3) y priorización Alta/Media de los requisitos

---

12. Criterios de aceptación del proyecto (Definition of Done)

1. La base de datos se crea ejecutando los cuatro scripts de `database/` sin errores y con integridad referencial.
2. El 100 % de las operaciones de la aplicación invocan procedimientos almacenados (verificable en el DAL: ninguna consulta T-SQL implementada en C#).
3. El mantenedor de insumos ejecuta alta, consulta con filtros, modificación y baja lógica con validaciones en UI y en BLL, y con mensajes funcionales ante errores de la base de datos.
4. Cada cambio en maestros y movimientos genera automáticamente un registro en `Auditoria` con usuario, fecha y valores anteriores/nuevos.
5. El módulo de reportes genera un PDF del log de auditoría según los filtros aplicados y un PDF de stock crítico.
6. La aplicación autentica, autoriza por rol, bloquea por intentos fallidos y registra los eventos de seguridad.
7. La solución compila sin errores y el repositorio contiene `/docs`, `/database` y `/src` con historial de commits incremental y descriptivo.

---

13. Conclusiones del análisis

El caso real de la bodega de insumos médicos permite demostrar de forma integral los contenidos de la asignatura: la problemática (planillas Excel compartidas, sin control de acceso ni trazabilidad) obliga primero a diseñar un modelo de datos normalizado y consistente, a especificar requisitos con criterios verificables y controles de seguridad explícitos, y solo después a construir la aplicación.

La decisión de diseño más relevante es **trasladar la seguridad y la trazabilidad al motor de datos** (procedimientos almacenados, restricciones de integridad y triggers de auditoría): la aplicación puede evolucionar sin perder el control de acceso ni la evidencia auditable, y el registro de cambios no queda a merced de un error en la interfaz. La arquitectura en cuatro capas hace sostenible ese esfuerzo: la UI no conoce SQL, la BLL concentra las reglas de negocio (incluida la validación de permisos) y el DAL es el único punto que invoca los SPs, de modo que el sistema resulta mantenible, verificable y auditable.

---

14. Anexos e índice de documentos

Documento	Contenido
`docs/01_Informe_Tecnico.md`	Este informe técnico
`docs/02_Requerimientos_RF_RNF_ISO27001.md`	RF con tríada, RNF y matriz de controles ISO 27001
`docs/03_Casos_de_Uso.md`	Diagrama UML de casos de uso y especificación textual
`docs/04_Diagrama_de_Clases.md`	Diagrama de clases UML (atributos, métodos y relaciones)
`docs/05_Modelo_Relacional_Normalizacion.md`	Modelo relacional, normalización 1FN→3FN e integridad referencial
`docs/06_Arquitectura_4_Capas.md`	Diseño estructural, responsabilidades y flujo de datos
`docs/07_Plan_de_Pruebas_y_Evidencias.md`	Casos de prueba, validaciones y evidencias
`docs/08_Manual_Ejecucion_Windows.md`	Manual de instalación y ejecución en Windows (entrega)
`docs/diagramas/`	Fuentes de los diagramas (Mermaid) usados en los documentos
`docs/evidencias/`	Capturas y PDF de muestra generados en la verificación

AIEP de la Universidad Andrés Bello

PARTE 2 DE 10

Especificación de Requerimientos

Requisitos funcionales con tríada, requisitos no funcionales y matriz ISO 27001

Documento 02 del repositorio (docs/02_Requerimientos_RF_RNF_ISO27001.md)

Ficha del documento	
Proyecto	SIGBIM — Sistema de Gestión de Bodega de Insumos Médicos
Asignatura	Desarrollo e Ingeniería de Software (PRO205)
Institución	AIEP — Universidad Andrés Bello
Estudiante	Jonathan Felipe Urbina Monasterio — `JONATHAN.URBINA@correoaiep.cl`
Documento	`docs/02_Requerimientos_RF_RNF_ISO27001.md` (Hito 1)
Fecha de emisión	septiembre de 2026

---

1. Convenciones de especificación

1.1 Identificación

Prefijo	Tipo	Prioridad
`RF-xx`	Requisito funcional	**Alta** (imprescindible), **Media** (deseable en el hito), **Baja** (evolución futura)
`RNF-xx`	Requisito no funcional	Clasificación: Seguridad, Rendimiento, Usabilidad, Mantenibilidad, Disponibilidad, Portabilidad
`CP-xx`	Caso de prueba derivado (ver `docs/07_Plan_de_Pruebas_y_Evidencias.md`)	—

1.2 Aplicación de la tríada (obligatoria)

Todo requisito funcional se redacta con la tríada:

> **[Rol / Usuario]** — *quién* ejecuta la acción **[Necesidad / Acción]** — *qué* necesita hacer y sobre qué objeto de negocio **[Criterio de aceptación / Seguridad]** — *cómo se verifica* objetivamente y *qué control* la protege

Notación del criterio: `DADO <precondición> CUANDO <acción> ENTONCES <resultado verificable>` y el control ISO/IEC 27001:2022 del Anexo A asociado.

1.3 Reglas de negocio transversales

Código	Regla
RN-01	Toda operación de datos se ejecuta mediante procedimientos almacenados (`CommandType.StoredProcedure`); se prohíbe SQL embebido en C#
RN-02	Ningún maestro se elimina físicamente: la baja es lógica (`Activo = 0`) y queda auditada
RN-03	El stock de un insumo nunca puede quedar negativo; una salida mayor al stock se rechaza con mensaje funcional
RN-04	El SKU del insumo y el RUT del proveedor son únicos (restricción `UNIQUE` en la base)
RN-05	Todo cambio sobre maestros y movimientos se registra automáticamente en `Auditoria` (trigger)
RN-06	Los registros de auditoría son de solo lectura para todos los roles; no existe operación de borrado
RN-07	Las contraseñas se almacenan derivadas (PBKDF2-SHA256 + sal) y nunca se escriben en el log ni en PDF
RN-08	Las actualizaciones usan control de concurrencia optimista (`RowVersion`): si otro usuario modificó el registro, la operación se rechaza

---

2. Requisitos funcionales

2.1 Módulo de seguridad (autenticación y autorización)

ID	Requisito	Tríada: Rol → Necesidad → Criterio
RF-01	**Inicio de sesión**	**Administrador, Bodeguero, Consulta** → Ingresar usuario y contraseña para acceder al sistema → DADO un usuario activo con credenciales correctas CUANDO presiona *Ingresar* ENTONCES se abre `FrmPrincipal` mostrando nombre y rol, se actualiza `UltimoAcceso` y se registra evento `LOGIN` en `Auditoria`. Control: A.5.17, A.8.5. Prioridad: Alta
RF-02	**Rechazo y bloqueo por intentos fallidos**	**Sistema** (en nombre del administrador) → Impedir ataques de fuerza bruta → DADO un usuario con 5 intentos fallidos consecutivos CUANDO intenta ingresar nuevamente ENTONCES ve el mensaje "Cuenta bloqueada temporalmente, intente en 15 minutos" y no accede; cada intento se registra como `LOGIN_FALLIDO`. Control: A.8.5, A.8.16. Prioridad: Alta
RF-03	**Autorización por rol y permisos**	**Todos los usuarios** → Acceder solo a las funciones que su rol autoriza → DADO un usuario del rol `Consulta` CUANDO abre el sistema ENTONCES los botones de crear/editar/eliminar están deshabilitados y, si se invoca la operación por otro medio, la BLL la rechaza y registra `ACCESO_DENEGADO`. Control: A.5.15, A.5.18, A.8.2, A.8.3. Prioridad: Alta

ID	Requisito	Tríada: Rol → Necesidad → Criterio
RF-04	**Gestión de usuarios (CRUD)**	**Administrador** → Crear, editar, activar/desactivar usuarios y asignarles un rol → DADO un administrador autenticado CUANDO crea un usuario con nombre de usuario o correo ya existente ENTONCES el SP lanza error controlado y la UI muestra "El nombre de usuario ya está registrado". Control: A.5.16, A.5.18. Prioridad: Alta
RF-05	**Gestión de roles y permisos**	**Administrador** → Asignar o quitar permisos funcionales a un rol → DADO un rol "Bodeguero" CUANDO se le quita el permiso `USUARIO_GESTIONAR` ENTONCES los usuarios de ese rol dejan de ver el módulo de seguridad y el intento de uso queda denegado y auditado. Control: A.5.15, A.8.2. Prioridad: Media
RF-06	**Cambio de contraseña con política**	**Todos los usuarios** → Cambiar su contraseña cumpliendo la política de robustez → DADO un usuario autenticado CUANDO ingresa una contraseña de menos de 10 caracteres o sin símbolo ENTONCES la BLL la rechaza; si cumple, se actualiza hash + sal, `DebeCambiarPassword = 0` y se registra el evento (sin exponer la clave). Control: A.5.17, A.8.24. Prioridad: Alta

2.2 Módulo de mantenedores (CRUD de la entidad principal y maestros)

ID	Requisito	Tríada: Rol → Necesidad → Criterio
RF-07	**CRUD de categorías de insumos**	**Bodeguero** → Clasificar los insumos por categoría → DADO un bodeguero autenticado CUANDO crea la categoría "Material de curación" y ya existe otra con ese nombre ENTONCES la base rechaza el duplicado (`UNIQUE`) y la UI informa "La categoría ya existe"; la creación exitosa queda en `Auditoria`. Control: A.8.28, A.8.15. Prioridad: Alta
RF-08	**CRUD de proveedores**	**Bodeguero** → Mantener el registro de proveedores con RUT, contacto y ciudad → DADO un proveedor con RUT `76.123.456-7` CUANDO se intenta registrar dos veces ENTONCES el segundo intento falla por `UNIQUE` con mensaje funcional; la modificación de datos de contacto queda auditada. Control: A.8.15. Prioridad: Alta
RF-09	**CRUD del insumo médico (entidad principal)**	**Bodeguero** → Crear, consultar, modificar y dar de baja insumos con SKU, categoría, proveedor, unidad de medida, precios, stock y stock mínimo → DADO un insumo "Jeringa 5 ml" creado por primera vez CUANDO se guarda ENTONCES aparece en la grilla con su `IdProducto`; si se repite el SKU falla con mensaje; la baja es lógica (`Activo = 0`), el registro no desaparece del log y la UI lo excluye de la lista. Control: A.8.28, A.8.15, RN-02, RN-04. Prioridad: Alta
RF-10	**Consulta con filtros de insumos**	**Bodeguero, Consulta** → Encontrar insumos por texto, categoría, proveedor o criticidad de stock → DADO 500 insumos cargados CUANDO se escribe "jering" y se selecciona la categoría ENTONCES la grilla devuelve solo las coincidencias activas en menos de 2 segundos (índices y SP con parámetros `@Texto`, `@IdCategoria`, `@IdProveedor`, `@SoloStockCritico`). Control: RNF-04. Prioridad: Alta
RF-11	**Stock crítico y vencimientos**	**Encargado de bodega** → Ver qué insumos requieren reposición o están por vencer → DADO un insumo con `Stock <= StockMinimo` CUANDO se activa el filtro "solo stock crítico" ENTONCES se listan exactamente los insumos que cumplen la condición, y los que vencen dentro de 90 días se marcan visualmente. Prioridad: Media
RF-12	**Registro de movimientos de stock**	**Bodeguero** → Registrar entradas (compra/reposición), salidas (entrega a boxes) y ajustes (merma, vencimiento) → DADO un insumo con stock 10 CUANDO se registra una salida de 15 ENTONCES el SP lanza error controlado ("Stock insuficiente: disponible 10") y no se graba nada (transacción); una salida válida actualiza `Producto.Stock`, inserta el movimiento con stock anterior/nuevo y ambos cambios quedan auditados. Control: RN-03, A.8.15. Prioridad: Alta
RF-13	**Consulta histórica de movimientos**	**Bodeguero, Encargado, Auditor** → Revisar los movimientos de un insumo en un rango de fechas → DADO un insumo con 30 movimientos CUANDO se consulta el rango 01-01 a 31-01 ENTONCES se listan en orden cronológico descendente con tipo, cantidad, usuario responsable y documento de referencia. Control: A.8.16. Prioridad: Media
RF-14	**Validaciones de datos en UI y BLL**	**Sistema** (en representación del negocio) → Impedir el ingreso de datos inválidos antes de llegar a la base → DADO el formulario de insumos CUANDO el usuario ingresa precio `-100`, stock `-5`, stock mínimo `0` o deja el nombre vacío ENTONCES no se permite guardar, se marca el campo en rojo con el motivo y no se invoca la base de datos. Control: RNF-11, A.8.28. Prioridad: Alta

2.3 Módulo de trazabilidad, auditoría y reportes

ID	Requisito	Tríada: Rol → Necesidad → Criterio
RF-15	**Auditoría automática de cambios (triggers)**	**Sistema (motor de base de datos)** → Registrar de forma automática e inalterable todo cambio en las tablas maestras → DADO un insumo con precio 1.500 CUANDO un bodeguero lo cambia a 1.800 ENTONCES se inserta en `Auditoria` una fila con operación `UPDATE`, tabla `Producto`, id del registro, usuario de aplicación, usuario de base, equipo, fecha y los JSON de valores anteriores y nuevos (precio 1500 → 1800); el registro no puede modificarse ni eliminarse desde la aplicación. Control: A.8.15, RN-05, RN-06. Prioridad: Alta
RF-16	**Registro de eventos de seguridad**	**Sistema** → Dejar evidencia de accesos y uso del sistema → DADO un intento de login con contraseña incorrecta CUANDO el sistema lo rechaza ENTONCES se inserta un evento `LOGIN_FALLIDO` con usuario y equipo, y el usuario no obtiene información sobre la existencia de la cuenta. Control: A.8.15, A.8.16. Prioridad: Alta
RF-17	**Consulta del log con filtros**	**Administrador, Auditor** → Investigar qué ocurrió y en qué registro → DADO un rango de fechas y el usuario `bod.central` CUANDO se consulta el log ENTONCES se listan solo los eventos de ese usuario en el rango, con la tabla, la operación, la fecha y el detalle, ordenados de más reciente a más antiguo. Control: A.8.16, RN-06. Prioridad: Alta
RF-18	**Reporte PDF del log de auditoría**	**Administrador, Auditor** → Obtener evidencia documental firmable del historial de cambios → DADO un conjunto de filtros aplicados en `FrmAuditoria` CUANDO se presiona *Generar PDF* ENTONCES se genera un archivo PDF en la carpeta de reportes con encabezado institucional, filtros utilizados, usuario emisor, fecha/hora de emisión, tabla de eventos y total de registros; adicionalmente se audita el evento `REPORTE_PDF` con la ruta del archivo generado. Control: A.8.16, A.8.12. Prioridad: Alta
RF-19	**Reporte PDF de stock crítico**	**Encargado de bodega** → Solicitar reposición con respaldo documental → DADO el listado de insumos con stock igual o inferior al mínimo CUANDO se genera el reporte ENTONCES el PDF muestra SKU, nombre, categoría, proveedor, stock actual, stock mínimo y cantidad sugerida de reposición. Control: A.8.16. Prioridad: Media
RF-20	**Cierre de sesión y control de sesión**	**Todos los usuarios** → Terminar su sesión de forma segura al ausentarse del puesto → DADO un usuario autenticado CUANDO presiona *Cerrar sesión* ENTONCES se registra el evento `LOGOUT`, se limpian los datos de sesión en memoria (incluidos permisos) y vuelve a `FrmLogin`. Control: A.5.15, A.8.15. Prioridad: Media

2.4 Módulo de persistencia, integridad y seguridad de datos

ID	Requisito	Tríada: Rol → Necesidad → Criterio
RF-21	**Toda operación vía procedimientos almacenados**	**Desarrollador** → Garantizar control y seguridad del acceso a datos → DADO el código de la solución CUANDO se revisa la capa DAL ENTONCES el 100 % de las operaciones usa `CommandType.StoredProcedure` con parámetros tipados y no existe SQL dinámico concatenado. Control: A.8.28, OWASP A03. Prioridad: Alta
RF-22	**Concurrencia optimista**	**Bodeguero, Administrador** → Evitar perder cambios cuando dos usuarios editan el mismo registro → DADO un insumo abierto por dos usuarios CUANDO el segundo guarda después del primero ENTONCES el SP detecta el `RowVersion` desactualizado y responde con el error controlado "El registro fue modificado por otro usuario, recargue la información". Control: RN-08. Prioridad: Media
RF-23	**Integridad referencial y validaciones en la base**	**Administrador de base de datos** → Impedir datos huérfanos o inconsistentes → DADO un intento de registrar un insumo con `IdCategoria` inexistente o una salida que dejaría stock negativo CUANDO se ejecuta el SP ENTONCES la operación se rechaza por `FOREIGN KEY` / `CHECK` y se devuelve un mensaje funcional a la UI. Control: RN-03, RN-04. Prioridad: Alta
RF-24	**Manejo de errores de base de datos en la aplicación**	**Todos los usuarios** → Recibir un mensaje comprensible y no un error técnico → DADO un error lanzado por un SP con `THROW` CUANDO la UI invoca la operación ENTONCES se muestra el mensaje funcional del negocio, se registra el detalle técnico en el log de la aplicación y esta permanece operativa (sin cierre inesperado). Control: RNF-11. Prioridad: Alta
RF-25	**Datos maestros parametrizables (unidades de medida)**	**Administrador** → Usar catálogos normalizados en lugar de texto libre → DADO el formulario de insumos CUANDO se selecciona la unidad de medida ENTONCES solo se ofrecen valores de la tabla `UnidadMedida` (`UN`, `CJ`, `CA`, `ML`, `GR`), evitando valores inconsistentes en el campo. Prioridad: Media

3. Requisitos no funcionales

ID	Categoría	Requisito	Criterio de verificación	Control ISO
RNF-01	Seguridad	Las contraseñas se almacenan derivadas con PBKDF2-SHA256, 100.000 iteraciones y sal aleatoria de 16 bytes por usuario	Revisión de `SeguridadService.DerivarClave` y de la tabla `Usuario`: ningún campo almacena la contraseña original	A.5.17, A.8.24
RNF-02	Seguridad	Ninguna credencial ni cadena de conexión queda en el código fuente ni en el repositorio	`git grep -i "password="` sin hallazgos; `appsettings.json` incluido en `.gitignore`	A.8.9, A.8.12, A.8.24
RNF-03	Seguridad / Trazabilidad	El 100 % de los cambios sobre maestros y movimientos queda auditado con usuario, fecha y valores anteriores/nuevos; retención en línea de 12 meses	Conteo comparativo entre operaciones ejecutadas y filas generadas en `Auditoria`	A.8.15
RNF-04	Rendimiento	La consulta del mantenedor responde en menos de 2 segundos con 50.000 insumos y 200.000 movimientos	Ejecución de `sp_Producto_Listar` con datos de volumen e índices sobre `Sku`, `Nombre`, `IdCategoria`, `IdProveedor`	—
RNF-05	Usabilidad	Interfaz en español, con validación en línea y formato local (CLP y fechas `dd-MM-yyyy`)	Revisión de los formularios y de la cultura `es-CL` configurada	—
RNF-06	Mantenibilidad	Arquitectura en 4 capas (UI, BLL, DAL, Entidades) con nomenclatura homogénea en español y sin dependencias invertidas	Inspección de referencias entre proyectos: UI→BLL→DAL→Entidades	—
RNF-07	Disponibilidad	Respaldo completo diario y de log, con procedimiento de restauración probado	Ejecución de `scripts/respaldo_bd.sql` y restauración en entorno de prueba	A.8.13
RNF-08	Portabilidad	Aplicación .NET 8 ejecutable en Windows 10/11 sobre SQL Server 2019 o superior, sin dependencias externas no administrables	`dotnet build` y `dotnet run` en el equipo Windows de entrega	—
RNF-09	Privacidad	El sistema no almacena datos clínicos de pacientes y el log no registra credenciales ni datos sensibles innecesarios	Revisión de columnas de `Auditoria` y del trigger de `Usuario` (enmascaramiento de hash y sal)	A.8.11, A.8.12
RNF-10	Seguridad	Los mensajes de error al usuario no revelan detalles técnicos (nombres de tablas, trazas o cadena de conexión)	Prueba de error forzado: se muestra el mensaje funcional y el técnico queda solo en el log de la aplicación	A.8.28, OWASP A09
RNF-11	Robustez	La aplicación no se cierra ante errores de base de datos ni ante datos inválidos del usuario	Ejecución de los casos de prueba negativos definidos en `docs/07`	—
RNF-12	Escalabilidad	Incorporar nuevos campos o entidades no debe requerir SQL nuevo en C#; se agrega un SP y su método en el DAL	Revisión de la arquitectura y prueba de extensión	—

4. Matriz de trazabilidad: requisitos ↔ controles ISO/IEC 27001:2022 ↔ evidencia

Control ISO 27001:2022	Requisitos que lo implementan	Evidencia verificable
A.5.15 Control de acceso	RF-03, RF-20	`SeguridadService.TienePermiso`; validación previa a cada operación de la BLL
A.5.16 Gestión de identidades	RF-04	Tabla `Usuario`; `FrmUsuarios` (alta, modificación, activación/desactivación)
A.5.17 Información de autenticación	RF-01, RF-06, RNF-01	Hash y sal almacenados en `Usuario`; derivación PBKDF2 en `SeguridadService`
A.5.18 Derechos de acceso	RF-04, RF-05	Tablas `Rol`, `Permiso`, `RolPermiso`; roles semilla por perfil
A.8.2 Privilegios de acceso	RF-03, RF-05	Permisos por rol; rol `Consulta` sin permisos de escritura

Control ISO 27001:2022	Requisitos que lo implementan	Evidencia verificable
A.8.3 Restricción de acceso a la información	RF-03	UI que deshabilita funciones no autorizadas y registro de `ACCESO_DENEGADO`
A.8.5 Autenticación segura	RF-01, RF-02, RF-06	Bloqueo a los 5 intentos (`IntentosFallidos`, `BloqueadoHasta`) y política de robustez
A.8.9 Gestión de configuraciones	RNF-02	`appsettings.json` externo, `appsettings.template.json` versionado, `.gitignore`
A.8.11 Enmascaramiento de datos	RNF-09	Trigger de `Usuario` que escribe `***` en `DatosAnteriores` / `DatosNuevos`
A.8.12 Prevención de fuga de información	RF-18, RNF-02, RNF-09	PDF sin credenciales; archivos de configuración fuera del repositorio
A.8.13 Copias de seguridad	RNF-07	`scripts/respaldo_bd.sql` y `database/04_seed_datos.sql`
A.8.15 Registro de eventos (logging)	RF-15, RF-16, RF-20, RNF-03	Triggers de auditoría + `sp_Log_Registrar` sobre la tabla `Auditoria`
A.8.16 Actividades de supervisión	RF-17, RF-18, RF-19	`FrmAuditoria` con filtros y reportes PDF
A.8.24 Uso de criptografía	RF-06, RNF-01, RNF-02	PBKDF2-SHA256 con sal; conexión configurable por variable de entorno
A.8.28 Codificación segura	RF-21, RF-23, RF-24, RNF-10	DAL con `CommandType.StoredProcedure` y parámetros tipados; `TRY/CATCH` + `THROW` en los SPs

4.1 Trazabilidad requisito → artefacto de implementación

Grupo de requisitos	Base de datos (`/database`)	Capa de la solución (`/src`)
RF-01 a RF-06 (seguridad)	`Usuario`, `Rol`, `Permiso`, `RolPermiso` + `sp_Usuario_*`, `sp_Rol_*`	`UI/FrmLogin`, `UI/FrmUsuarios`, `BLL/SeguridadService`, `DAL/UsuarioDAL`
RF-07 a RF-11 (mantenedores)	`Categoria`, `Proveedor`, `Producto`, `UnidadMedida` + `sp_Categoria_*`, `sp_Proveedor_*`, `sp_Producto_*`	`UI/FrmProductos`, `UI/FrmCategorias`, `UI/FrmProveedores`, `BLL/*Service`, `DAL/*DAL`
RF-12 y RF-13 (movimientos)	`MovimientoStock` + `sp_Movimiento_*`	`UI/FrmMovimientos`, `BLL/MovimientoStockService`, `DAL/MovimientoStockDAL`
RF-14 (validaciones)	Restricciones `CHECK` / `UNIQUE` / `FK`	`UI/ValidadorFormulario`, `BLL/Validador`
RF-15 a RF-20 (trazabilidad)	`Auditoria` + triggers + `sp_Log_*`	`UI/FrmAuditoria`, `BLL/AuditoriaService`, `DAL/AuditoriaDAL`
RF-18 y RF-19 (reportes PDF)	`sp_Reporte_Auditoria`, `sp_Reporte_StockCritico`	`Reportes/ReportePdfServicio`, `BLL/ReporteService`
RF-21 a RF-25 (persistencia e integridad)	Todos los SPs, índices y columnas `RowVersion`	`DAL/ConexionSql`, `DAL/MapeadorSql`, `BLL/Validador`

AIEP de la Universidad Andrés Bello

PARTE 3 DE 10

Modelo de Casos de Uso (UML)

Interacciones de los actores con las funciones del mantenedor

`Documento 03 del repositorio (docs/03_Casos_de_Uso.md)`

Ficha del documento	
Proyecto	SIGBIM — Sistema de Gestión de Bodega de Insumos Médicos
Asignatura	Desarrollo e Ingeniería de Software (PRO205)
Institución	AIEP — Universidad Andrés Bello
Estudiante	Jonathan Felipe Urbina Monasterio — `JONATHAN.URBINA@correoaiep.cl`
Documento	`docs/03_Casos_de_Uso.md` (Hito 1)
Fecha de emisión	septiembre de 2026

Modelado UML 2.5 de las interacciones entre los actores y las funciones del mantenedor (CRUD), la seguridad y la trazabilidad. Los diagramas se escriben en **Mermaid** para que GitHub los renderice directamente y para que permanezcan versionados junto al código.

---

1. Actores

Actor	Tipo	Descripción
Administrador	Humano (primario)	Responsable de la seguridad: gestiona usuarios, roles y permisos; consulta el log
Bodeguero	Humano (primario)	Usuario operativo: CRUD de insumos y maestros, registro de movimientos de stock
Consulta (encargado / auditor)	Humano (primario)	Solo lectura: consulta existencias, stock crítico, log y reportes PDF
Motor SQL Server	Sistema (secundario)	Ejecuta los SPs, aplica las restricciones de integridad y dispara los triggers de auditoría

---

2. Diagrama de casos de uso (general)

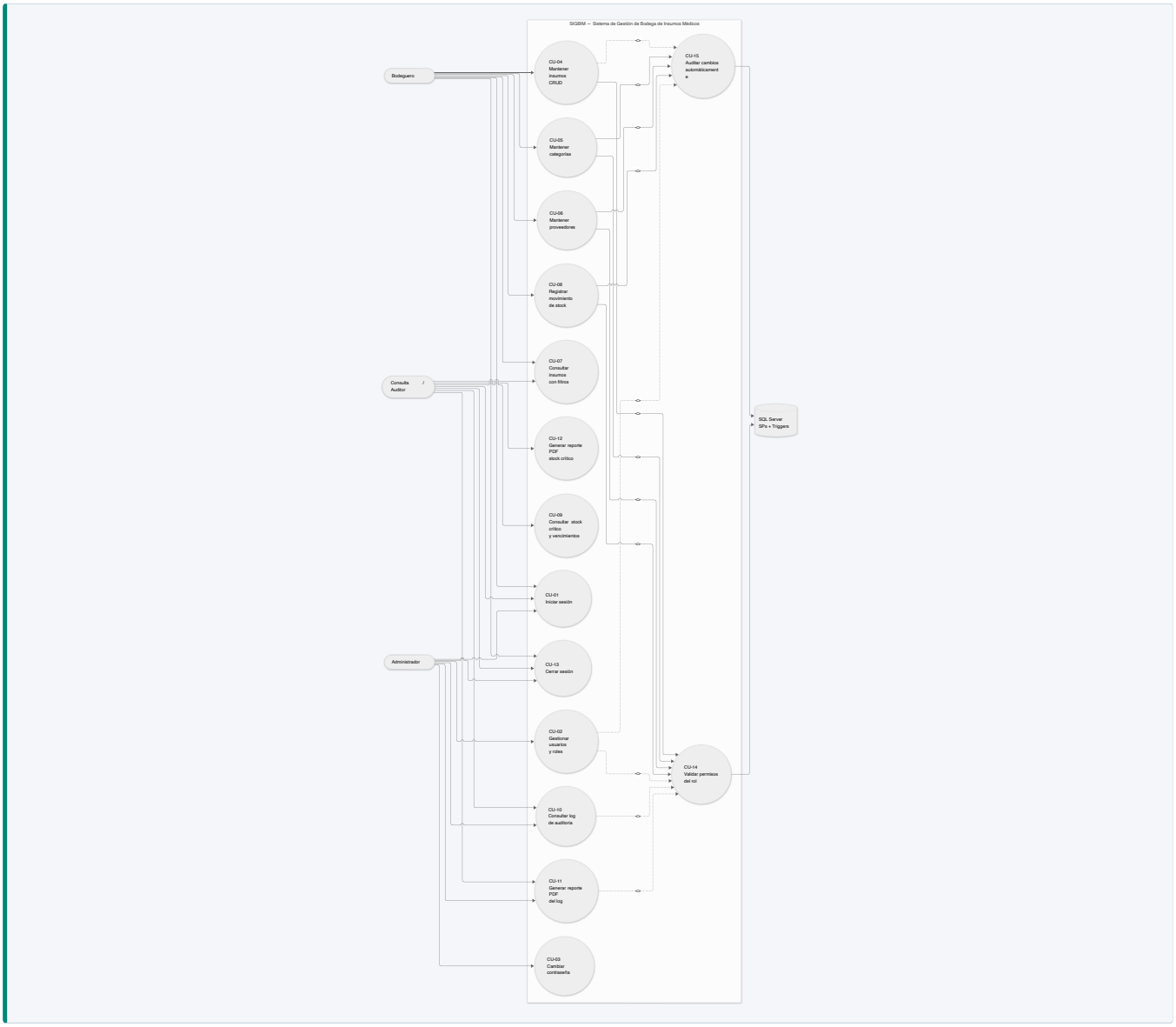


Figura 1 — Diagrama del documento (fuente Mermaid versionada en el repositorio).

2.1 Casos de uso del CRUD de la entidad principal (`Producto`)

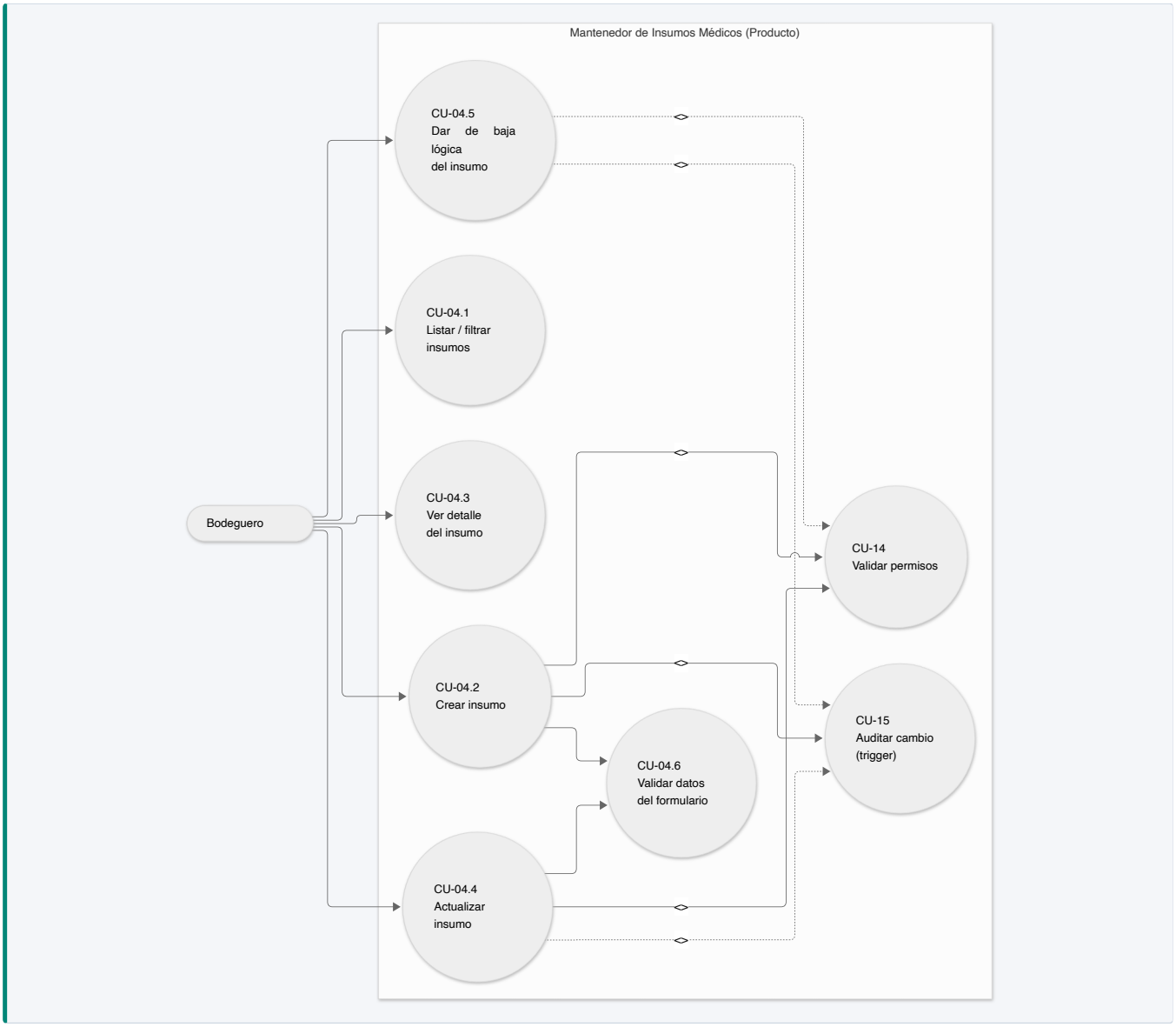


Figura 2 — Diagrama del documento (fuente Mermaid versionada en el repositorio).

3. Especificación textual de casos de uso

CU	Nombre	Actor(es)	Precondición	Flujo principal (resumen)	Flujo alterno / excepciones	Postcondición
CU-01	Iniciar sesión	Todos	Aplicación abierta	Ingresa usuario y contraseña → BLL valida formato → `sp_Usuario_ObtenerPorNombre` → verifica PBKDF2 → `sp_Usuario_RegistrarAcceso` → abre `FrmPrincipal`	Credenciales inválidas → `sp_Usuario_RegistrarIntentoFallido` y mensaje genérico; 5 fallos → cuenta bloqueada 15 minutos; usuario inactivo → acceso denegado	Sesión iniciada, `UltimoAcceso` actualizado y evento `LOGIN` auditado
CU-02	Gestionar usuarios y roles	Administrador	Sesión con permiso `USUARIO_GESTIONAR`	Lista usuarios → crea / edita / activa-desactiva → asigna rol → SP correspondiente	Usuario o correo duplicado → error funcional; sin permiso → `ACCESO_DENEGADO` auditado	Usuario persistido y auditado por trigger
CU-03	Cambiar contraseña	Todos	Sesión activa	Ingresa contraseña actual y nueva → BLL valida política → nueva sal + hash → `sp_Usuario_CambiarPassword`	Contraseña actual incorrecta; la nueva no cumple la política o es igual a la anterior	Contraseña actualizada `DebeCambiarPasswo = 0` y evento auditado
CU-04	Mantener insumos (CRUD)	Bodeguero	Sesión con permiso `PRODUCTO_*`	Lista con filtros → crea / edita / da de baja con validaciones en UI y BLL → invoca los SPs	SKU duplicado; categoría o proveedor inexistente; precio/stock inválidos; `RowVersion` desactualizado (concurrencia)	Registro creado, actualizado o inactivo, con el cambio auditado en JSON
CU-05	Mantener categorías	Bodeguero	Sesión con permiso `CATEGORIA_*`	CRUD con validación de nombre único	Nombre duplicado; categoría con insumos activos al intentar la baja	Categoría persistida y auditada

CU	Nombre	Actor(es)	Precondición	Flujo principal (resumen)	Flujo alterno / excepciones	Postcondición
CU-06	Mantener proveedores	Bodeguero	Sesión con permiso `PROVEEDOR_*`	CRUD con validación de RUT y correo	RUT duplicado; formato de correo inválido	Proveedor persistido y auditado
CU-07	Consultar insumos con filtros	Bodeguero, Consulta / Auditor	Sesión activa	Define texto, categoría, proveedor y "solo stock crítico" → `sp_Producto_Listar` → grilla	Sin resultados → grilla vacía con mensaje informativo	Datos desplegados (so lectura para el rol Consulta)
CU-08	Registrar movimiento de stock	Bodeguero	Sesión con permiso `MOVIMIENTO_REGISTRAR`	Selecciona insumo, tipo, cantidad, motivo y documento → `sp_Movimiento_Registrar` (transacción)	Salida mayor al stock disponible → error "Stock insuficiente" sin cambios; insumo inactivo → rechazo	Stock actualizado, movimiento insertado y ambos auditados
CU-09	Consultar stock crítico y vencimientos	Bodeguero, Consulta / Auditor	Sesión activa	Filtra insumos con stock ≤ mínimo o por vencer dentro de 90 días	—	Listado visible y exportable a PDF (CU-
CU-10	Consultar log de auditoría	Administrador, Consulta / Auditor	Permiso `LOG_VER`	Aplica filtros (rango, usuario, tabla, operación, texto) → `sp_Log_Listar` → grilla de solo lectura	Rango inválido (desde > hasta) → validación en UI	Eventos desplegados, s posibilidad de edición
CU-11	Generar reporte PDF del log	Administrador, Consulta / Auditor	Permiso `REPORTE_PDF` y resultado de CU-10	Presiona *Generar PDF* → la BLL obtiene datos y metadatos → `ReportePdfServicio` crea el PDF → registra el evento `REPORTE_PDF`	Carpeta de salida sin permisos → mensaje funcional y opción de elegir otra ubicación	PDF con filtros, emisor totales; evento auditad
CU-12	Generar reporte PDF de stock crítico	Bodeguero, Consulta / Auditor	Permiso `REPORTE_PDF`	Genera el reporte con la sugerencia de reposición → PDF	Sin insumos críticos → PDF informativo "sin registros"	PDF generado y evento auditado
CU-13	Cerrar sesión	Todos	Sesión activa	Presiona *Cerrar sesión* → registra `LOGOUT` → limpia los datos de sesión → vuelve a `FrmLogin`	—	Sesión finalizada y auditada
CU-14	Validar permisos del rol	Sistema	Operación solicitada	La BLL consulta los permisos del rol del usuario y autoriza o rechaza	Sin permiso → excepción de negocio y registro de `ACCESO_DENEGADO`	Operación autorizada o rechazada (siempre trazable)
CU-15	Auditar cambios automáticamente	Sistema (SQL Server)	Operación DML ejecutada	El trigger captura valores anteriores y nuevos, usuario, equipo y fecha → inserta en `Auditoria`	—	Evidencia disponible pa CU-10 y CU-11

4. Matriz actor ↔ caso de uso

Caso de uso	Administrador	Bodeguero	Consulta / Auditor	SQL Server
CU-01 Iniciar sesión	✔	✔	✔	
CU-02 Gestionar usuarios y roles	✔			
CU-03 Cambiar contraseña	✔	✔	✔	
CU-04 Mantener insumos (CRUD)	✔	✔		
CU-05 Mantener categorías		✔		
CU-06 Mantener proveedores		✔		
CU-07 Consultar insumos con filtros		✔	✔	
CU-08 Registrar movimiento de stock		✔		
CU-09 Consultar stock crítico y vencimientos		✔	✔	
CU-10 Consultar log de auditoría	✔		✔	
CU-11 Generar reporte PDF del log	✔		✔	
CU-12 Generar reporte PDF de stock crítico		✔	✔	
CU-13 Cerrar sesión	✔	✔	✔	
CU-14 Validar permisos del rol				✔
CU-15 Auditar cambios automáticamente				✔

5. Relaciones `<<include>>` y `<<extend>>` justificadas

Relación	Casos de uso	Justificación
`<<include>>`	CU-04, CU-05, CU-06, CU-02, CU-08, CU-10, CU-11 → CU-14	Toda operación de negocio exige la validación previa de permisos del rol (control de acceso, ISO 27001 A.5.15/A.8.2); es obligatoria y no opcional
`<<include>>`	CU-04, CU-05, CU-06, CU-08, CU-02 → CU-15	Toda operación DML se audita de manera automática en el motor (A.8.15)
`<<extend>>`	CU-02 → asignación de permisos al rol	La asignación o modificación de los permisos de un rol (tabla `RolPermiso`) se ejecuta solo cuando el administrador decide ajustar el RBAC; es un paso opcional dentro de la gestión de usuarios
`<<extend>>`	CU-10 → CU-11 / CU-12	El reporte PDF se genera solo si el usuario decide exportar el resultado de la consulta
`<<extend>>`	CU-01 → CU-03	El cambio de contraseña obligatorio se ejecuta cuando `DebeCambiarPassword = 1`, como extensión del inicio de sesión



PARTE 4 DE 10

Diagrama de Clases (UML)

Atributos, visibilidad, métodos, relaciones y mapeo a las capas

`Documento 04 del repositorio (docs/04_Diagrama_de_Clases.md)`

Ficha del documento	
Proyecto	SIGBIM — Sistema de Gestión de Bodega de Insumos Médicos
Asignatura	Desarrollo e Ingeniería de Software (PRO205)
Institución	AIEP — Universidad Andrés Bello
Estudiante	Jonathan Felipe Urbina Monasterio — `JONATHAN.URBINA@correoaiep.cl`
Documento	`docs/04_Diagrama_de_Clases.md` (Hito 1)
Fecha de emisión	septiembre de 2026

Modelo de clases del dominio del mantenedor de insumos médicos, con atributos, visibilidad, métodos principales y relaciones. Es la base del modelo relacional documentado en `docs/05_Modelo_Relacional_Normalizacion.md` y de las entidades de la capa `PRO205.Entidades`.

---

1. Diagrama de clases del dominio

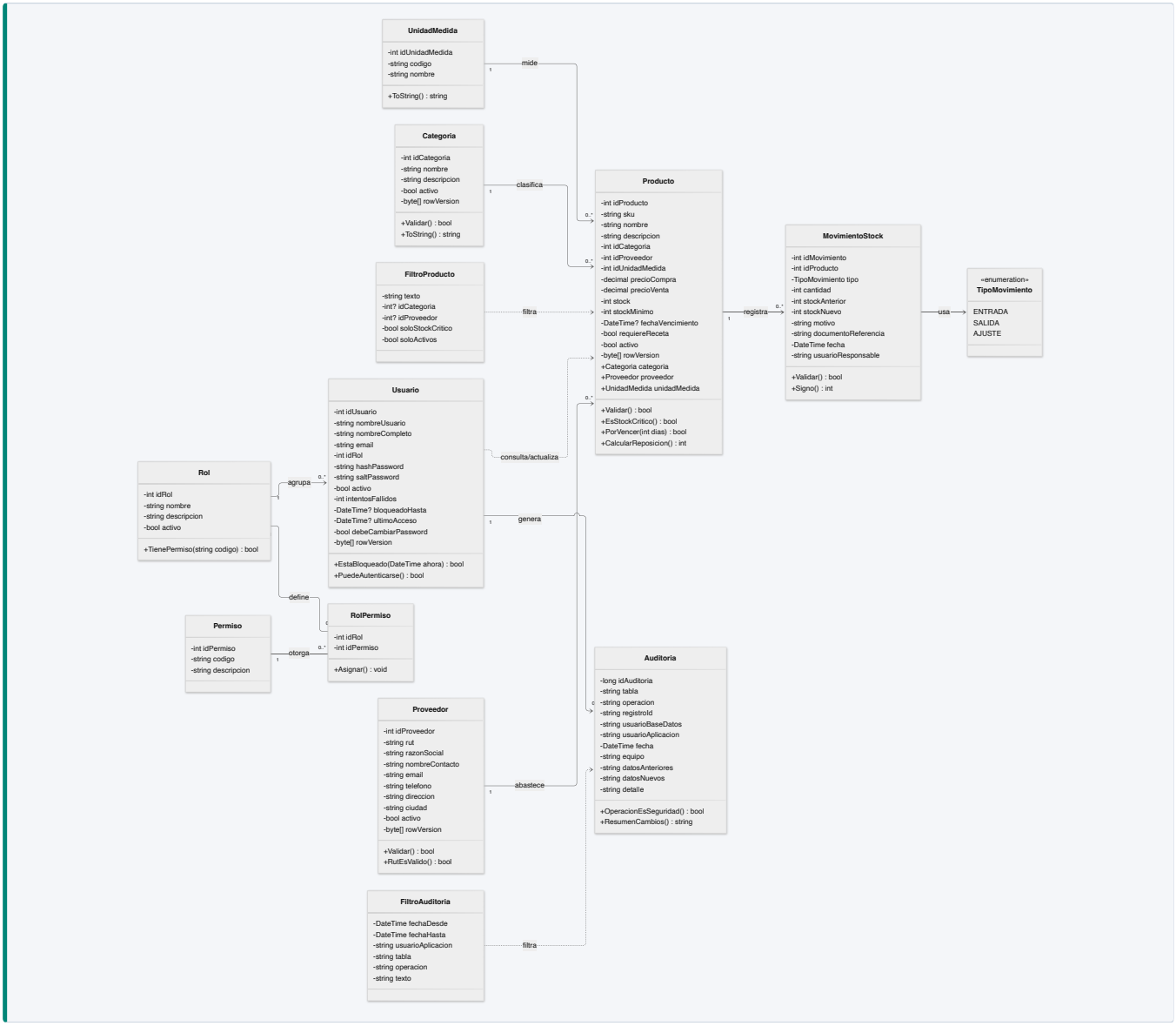


Figura 1 — Diagrama del documento (fuente Mermaid versionada en el repositorio).

---

2. Descripción de las clases del dominio

Clase	Responsabilidad	Atributos clave	Métodos principales	Relaciones
`Categoria`	Clasificar los insumos del catálogo	`idCategoria`, `nombre` (único), `descripcion`, `activo`, `rowVersion`	`Validar()`, `ToString()`	1 → N con `Producto`

Clase	Responsabilidad	Atributos clave	Métodos principales	Relaciones
`Proveedor`	Registrar los datos del distribuidor de insumos	`rut` (único), `razonSocial`, `nombreContacto`, `email`, `telefono`, `ciudad`, `activo`	`Validar()`, `RutEsValido()`	1 → N con `Producto`
`UnidadMedida`	Tabla de referencia de unidades (UN, CJ, CA, ML, GR)	`idUnidadMedida`, `codigo` (único), `nombre`	`ToString()`	1 → N con `Producto`
`Producto` (**entidad principal**)	Representar el insumo médico y su inventario	`sku` (único), `nombre`, `precioCompra`, `precioVenta`, `stock`, `stockMinimo`, `fechaVencimiento`, `activo`, `rowVersion`	`Validar()`, `EsStockCritico()`, `PorVencer(dias)`, `CalcularReposicion()`	N → 1 con `Categoria`, `Proveedor`, `UnidadMedida`; 1 → N con `MovimientoStock`
`MovimientoStock`	Registrar entradas, salidas y ajustes con el stock resultante	`tipo`, `cantidad`, `stockAnterior`, `stockNuevo`, `motivo`, `documentoReferencia`, `fecha`, `usuarioResponsable`	`Validar()`, `Signo()`	N → 1 con `Producto`; usa la enumeración `TipoMovimiento`
`Usuario`	Representar la identidad y el estado de credenciales	`nombreUsuario` (único), `email` (único), `hashPassword`, `saltPassword`, `intentosFallidos`, `bloqueadoHasta`, `ultimoAcceso`, `debeCambiarPassword`, `activo`	`EstaBloqueado(ahora)`, `PuedeAutenticarse()`	N → 1 con `Rol`; 1 → N con `Auditoria`
`Rol`	Agrupar permisos por perfil de usuario	`nombre` (único), `descripcion`, `activo`	`TienePermiso(codigo)`	1 → N con `Usuario`; N ↔ N con `Permiso` vía `RolPermiso`
`Permiso`	Definir una capacidad funcional atómica	`codigo` (único), `descripcion`	—	N ↔ N con `Rol` vía `RolPermiso`
`RolPermiso`	Resolver la relación N:M del RBAC	`idRol`, `idPermiso` (PK compuesta)	`Asignar()`	N → 1 con `Rol` y con `Permiso`
`Auditoria`	Almacenar la evidencia de cada cambio y evento de seguridad	`tabla`, `operacion`, `registroId`, `usuarioBaseDatos`, `usuarioAplicacion`, `fecha`, `equipo`, `datosAnteriores`, `datosNuevos`, `detalle`	`OperacionEsSeguridad()`, `ResumenCambios()`	N → 1 con `Usuario` (de referencia)
`FiltroProducto`	DTO de criterios de búsqueda del mantenedor	`texto`, `idCategoria`, `idProveedor`, `soloStockCritico`, `soloActivos`	—	Depende de `Producto`
`FiltroAuditoria`	DTO de criterios de consulta del log	`fechaDesde`, `fechaHasta`, `usuarioAplicacion`, `tabla`, `operacion`, `texto`	—	Depende de `Auditoria`

2.1 Multiplicidad y reglas de integridad derivadas del diagrama

Relación	Multiplicidad	Implementación relacional
`Categoria` — `Producto`	1 : 0..*	FK `Producto.IdCategoria` → `Categoria.IdCategoria` (obligatoria)
`Proveedor` — `Producto`	1 : 0..*	FK `Producto.IdProveedor` → `Proveedor.IdProveedor` (obligatoria)
`UnidadMedida` — `Producto`	1 : 0..*	FK `Producto.IdUnidadMedida` → `UnidadMedida.IdUnidadMedida` (obligatoria)
`Producto` — `MovimientoStock`	1 : 0..*	FK `MovimientoStock.IdProducto` → `Producto.IdProducto` (obligatoria)
`Rol` — `Usuario`	1 : 0..*	FK `Usuario.IdRol` → `Rol.IdRol` (obligatoria)
`Rol` — `Permiso`	0..* : 0..*	Tabla intermedia `RolPermiso` con PK compuesta (resuelve el N:M)
`Usuario` — `Auditoria`	1 : 0..*	Columna `UsuarioAplicacion` (varchar, referencia lógica por nombre de usuario)

3. Reglas de negocio expresadas en los métodos del dominio

Método	Regla que encapsula	Validación equivalente en base de datos
`Producto.Validar()`	Nombre obligatorio (3–120), SKU obligatorio (3–20), precios ≥ 0, `stock ≥ 0`, `stockMinimo ≥ 0`, categoría/proveedor/unidad obligatorios	`UNIQUE` en `Sku`, `CHECK (PrecioCompra >= 0)`, `CHECK (Stock >= 0)`, `CHECK (StockMinimo >= 0)`, `FOREIGN KEY`
`Producto.EsStockCritico()`	Un insumo está crítico si `stock <= stockMinimo`	Filtro `@SoloStockCritico` dentro de `sp_Producto_Listar`
`Producto.PorVencer(dias)`	Marca alerta si `fechaVencimiento <= hoy + 90 días`	Filtro de vencimiento en `sp_Producto_Listar` y `sp_Reporte_StockCritico`
`Producto.CalcularReposicion()`	Sugerencia de reposición = `(stockMinimo * 2) − stock` (mínimo 0)	Cálculo expuesto por `sp_Reporte_StockCritico`

Método	Regla que encapsula	Validación equivalente en base de datos
`MovimientoStock.Validar()`	Cantidad > 0, motivo obligatorio, tipo válido, `stockNuevo = stockAnterior ± cantidad` y `stockNuevo >= 0`	`CHECK (Cantidad > 0)`, `CHECK (Tipo IN (...))`, validación con `THROW` dentro de `sp_Movimiento_Registrar`
`Usuario.EstaBloqueado(ahora)`	Bloqueado si `bloqueadoHasta > ahora`	Columnas `IntentosFallidos` y `BloqueadoHasta` con `CHECK (IntentosFallidos >= 0)`
`Rol.TienePermiso(codigo)`	El usuario solo ejecuta acciones incluidas en los permisos de su rol	`sp_Rol_ObtenerPermisos` + validación en `SeguridadService`
`Auditoria.OperacionEsSeguridad()`	Identifica eventos de seguridad (`LOGIN` , `LOGIN_FALLIDO` , `ACCESO_DENEGADO` , `REPORTE_PDF` , `LOGOUT`)	`CHECK (Operacion IN (...))` en `Auditoria`

4. Diagrama de clases de la arquitectura (mapeo a las capas)

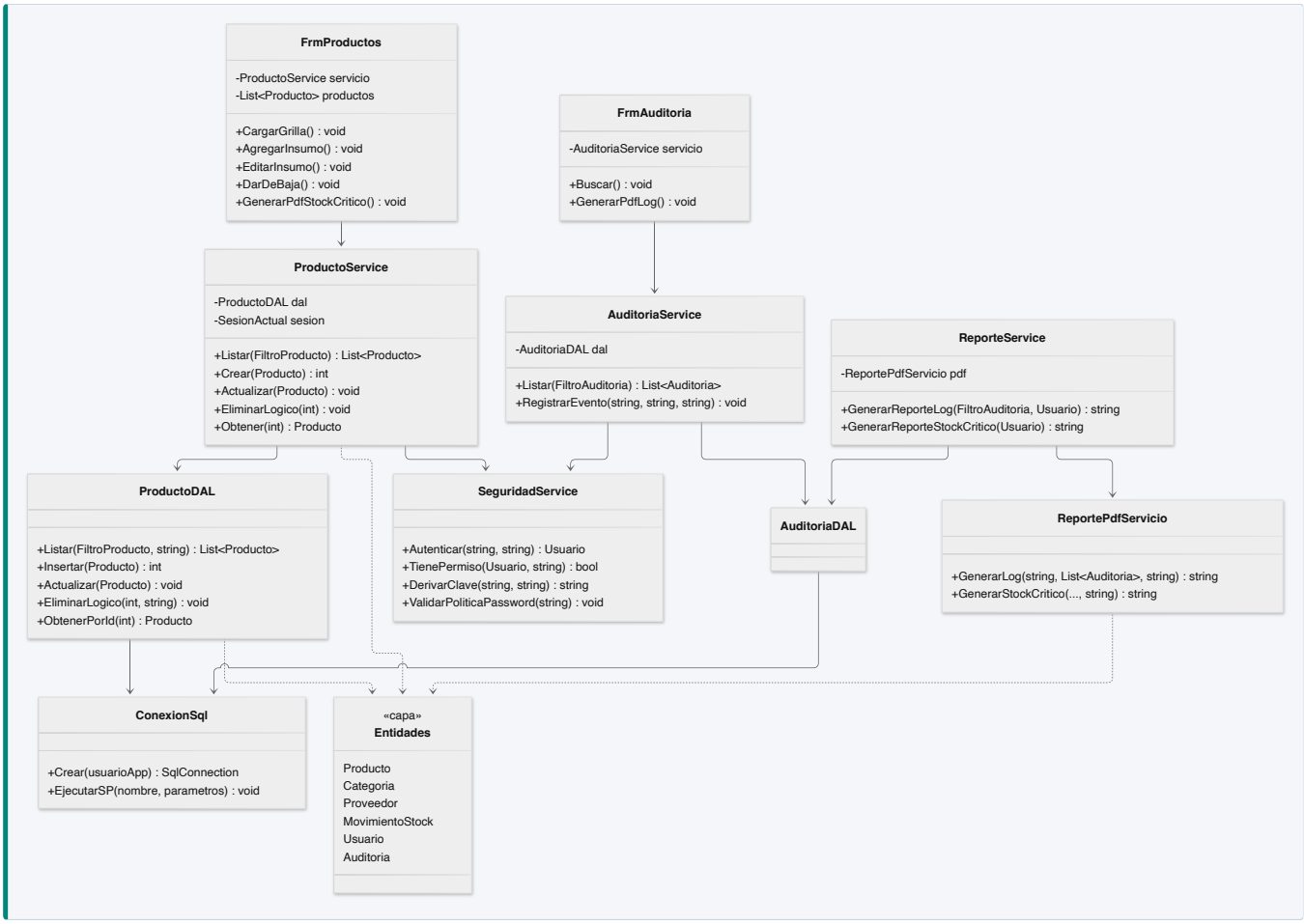


Figura 2 — Diagrama del documento (fuente Mermaid versionada en el repositorio).

5. Correspondencia clase de dominio ↔ tabla del modelo relacional

Clase UML	Tabla relacional	Observación
`Categoria`	`Categoria`	Maestro con baja lógica (`Activo`)
`Proveedor`	`Proveedor`	`Rut UNIQUE` ; maestro con baja lógica
`UnidadMedida`	`UnidadMedida`	Tabla de referencia (3FN: evita texto libre en `Producto`)
`Producto`	`Producto`	Entidad principal; FK a los tres maestros + `RowVersion`
`MovimientoStock`	`MovimientoStock`	Histórico inmutable de movimientos
`TipoMovimiento` (enum)	Restricción `CHECK`	`CHECK (Tipo IN ('ENTRADA','SALIDA','AJUSTE'))`
`Usuario`	`Usuario`	Credenciales derivadas + control de bloqueo
`Rol` / `Permiso` / `RolPermiso`	`Rol` / `Permiso` / `RolPermiso`	El N:M se resuelve con tabla intermedia
`Auditoria`	`Auditoria`	Poblada por triggers y `sp_Log_Registrar`

Clase UML	Tabla relacional	Observación
`FiltroProducto` / `FiltroAuditoria`	—	Clases de transferencia (DTO) traducidas a parámetros de los SPs

PARTE 5 DE 10

Modelo Relacional y Normalización

Transformación a 1FN, 2FN y 3FN con integridad referencial

`Documento 05 del repositorio (docs/05_Modelo_Relacional_Normalizacion.md)`

Ficha del documento	
Proyecto	SIGBIM — Sistema de Gestión de Bodega de Insumos Médicos
Asignatura	Desarrollo e Ingeniería de Software (PRO205)
Institución	AIEP — Universidad Andrés Bello
Estudiante	Jonathan Felipe Urbina Monasterio — `JONATHAN.URBINA@correoaiep.cl`
Documento	`docs/05_Modelo_Relacional_Normalizacion.md` (Hito 1)
Fecha de emisión	septiembre de 2026

Transformación del diagrama de clases (`docs/04_Diagrama_de_Clases.md`) al modelo relacional normalizado (1FN, 2FN, 3FN) con integridad referencial, tal como se implementa en `database/01_esquema.sql`.

---

1. Modelo relacional (notación)

```
Rol            (IdRol PK, Nombre UQ, Descripcion, Activo, FechaCreacion, UsuarioCreacion)
Permiso        (IdPermiso PK, Codigo UQ, Descripcion)
RolPermiso     (IdRol PK/FK→Rol, IdPermiso PK/FK→Permiso)
Usuario        (IdUsuario PK, NombreUsuario UQ, NombreCompleto, Email UQ, IdRol FK→Rol,
                HashPassword, SaltPassword, Activo, IntentosFallidos, BloqueadoHasta,
                UltimoAcceso, DebeCambiarPassword, FechaCreacion, UsuarioCreacion,
                FechaModificacion, UsuarioModificacion, RowVersion)

Categoria      (IdCategoria PK, Nombre UQ, Descripcion, Activo, FechaCreacion, UsuarioCreacion,
                FechaModificacion, UsuarioModificacion, RowVersion)
Proveedor      (IdProveedor PK, Rut UQ, RazonSocial, NombreContacto, Email, Telefono, Direccion,
                Ciudad, Activo, FechaCreacion, UsuarioCreacion, FechaModificacion,
                UsuarioModificacion, RowVersion)
UnidadMedida   (IdUnidadMedida PK, Codigo UQ, Nombre)

Producto       (IdProducto PK, Sku UQ, Nombre, Descripcion, IdCategoria FK→Categoria,
                IdProveedor FK→Proveedor, IdUnidadMedida FK→UnidadMedida, PrecioCompra,
                PrecioVenta, Stock, StockMinimo, FechaVencimiento, RequiereReceta, Activo,
                FechaCreacion, UsuarioCreacion, FechaModificacion, UsuarioModificacion, RowVersion)

MovimientoStock(IdMovimiento PK, IdProducto FK→Producto, Tipo, Cantidad, StockAnterior, StockNuevo,
                Motivo, DocumentoReferencia, Fecha, UsuarioResponsable)

Auditoria      (IdAuditoria PK, Tabla, Operacion, RegistroId, UsuarioBaseDatos, UsuarioAplicacion,
                Fecha, Equipo, DatosAnteriores, DatosNuevos, Detalle)
```

Leyenda: `PK` = llave primaria, `FK` = llave foránea, `UQ` = restricción `UNIQUE`.

---

2. Normalización (1FN → 3FN)

2.1 Punto de partida (planilla Excel del proceso actual — sin normalizar)

Fila	Insumo	Categoría	Proveedor (RUT – razón social – contacto – ciudad)	Unidad	Precio	Stock	¿Crítico?	Movimientos (fecha/cantidad/motivo)
1	Jeringa 5 ml	Insumo desechable	76.123.456-7 – Distribuidora Médica Sur – Ana Pérez – Santiago	Unidad	120	40	Sí	12-09/ENTRADA/200; 14-09/SALIDA/160
2	Jeringa 5 ml	Insumo desechable	76.123.456-7 – Distribuidora Médica Sur – Ana Pérez – Santiago	Unidad	120	40	Sí	15-09/SALIDA/20

Problemas detectados: datos repetidos (categoría, proveedor y sus atributos), listas de movimientos dentro de una celda, mezcla de datos del insumo con datos del proveedor, columna calculada `¿Crítico?`, imposibilidad de distinguir movimientos y ausencia de identificadores estables.

2.2 Primera Forma Normal (1FN): atomicidad de los atributos

- La celda "Movimientos" con varios valores se extrae a la tabla `MovimientoStock`, una fila por movimiento.
- El campo compuesto "Proveedor" (`RUT – razón social – contacto – ciudad`) se descompone en columnas atómicas dentro de `Proveedor`.
- Se incorpora un identificador único por entidad (`IdProducto`, `IdProveedor`, etc.), eliminando la dependencia de valores descriptivos.

2.3 Segunda Forma Normal (2FN): dependencia funcional total de la llave primaria

- En `RolPermiso` (PK compuesta `IdRol` + `IdPermiso`) la asignación de un permiso depende de **ambos** atributos: no existen atributos parcialmente dependientes.
- En `MovimientoStock` todos los atributos (`Tipo`, `Cantidad`, `Motivo`, `Fecha` …) dependen por completo de `IdMovimiento`.
- Los datos de proveedor y categoría **no** se repiten en `Producto`: se referencian por `IdProveedor` / `IdCategoria`, evitando dependencias parciales respecto de la descripción del insumo.

2.4 Tercera Forma Normal (3FN): eliminación de dependencias transitivas

Dependencia transitiva detectada	Resolución aplicada (3FN)
`Producto` guardaba el nombre de la categoría (dependía de la categoría, no del insumo)	Se crea `Categoria` y `Producto.IdCategoria` como FK: el nombre de la categoría queda solo en su tabla
`Producto` guardaba la razón social, contacto y ciudad del proveedor	Se crea `Proveedor` y `Producto.IdProveedor` como FK; RUT con `UNIQUE`
`Producto` guardaba la unidad como texto libre ("unidad", "caja", "ml")	Se crea la tabla de referencia `UnidadMedida` (`Codigo` `UNIQUE`) y `Producto.IdUnidadMedida` como FK
`MovimientoStock` guardaba el nombre del insumo y su categoría	Solo se almacena `IdProducto` (FK); la descripción se obtiene por `JOIN` en la vista `vw_ProductoDetalle`
`Usuario` guardaba el nombre del rol y la lista de permisos en texto	Se crean `Rol` , `Permiso` y `RolPermiso` ; el usuario solo referencia `IdRol`
Columna calculada "¿Crítico?" y "cantidad sugerida"	No se almacenan: se calculan al consultar (`Stock <= StockMinimo` , `(StockMinimo * 2) - Stock`)

Resultado: ninguna tabla presenta atributos multivaluados (1FN), ningún atributo no clave depende parcialmente de la llave primaria (2FN) y ningún atributo no clave depende de otro atributo no clave (3FN).

---

3. Integridad referencial y restricciones de integridad

3.1 Llaves foráneas

Tabla hija	Columna	Tabla padre	Columna	ON DELETE / ON UPDATE
`Usuario`	`IdRol`	`Rol`	`IdRol`	`NO ACTION` (impide eliminar un rol en uso)
`RolPermiso`	`IdRol`	`Rol`	`IdRol`	`CASCADE` (al borrar un rol, se liberan sus asignaciones)
`RolPermiso`	`IdPermiso`	`Permiso`	`IdPermiso`	`CASCADE`
`Producto`	`IdCategoria`	`Categoria`	`IdCategoria`	`NO ACTION` (obliga a baja lógica, RN-02)
`Producto`	`IdProveedor`	`Proveedor`	`IdProveedor`	`NO ACTION`
`Producto`	`IdUnidadMedida`	`UnidadMedida`	`IdUnidadMedida`	`NO ACTION`
`MovimientoStock`	`IdProducto`	`Producto`	`IdProducto`	`NO ACTION` (protege el histórico)

3.2 Restricciones `UNIQUE` , `CHECK` y `DEFAULT`

Tabla	Restricción	Definición
`Usuario`	`UQ_Usuario_NombreUsuario` , `UQ_Usuario_Email`	Unicidad de credencial y correo
`Usuario`	`CK_Usuario_Intentos`	`IntentosFallidos >= 0`
`Usuario`	`CK_Usuario_Email`	`Email LIKE '%_@_%._%'`
`Categoria`	`UQ_Categoria_Nombre`	`Nombre UNIQUE`
`Proveedor`	`UQ_Proveedor_Rut`	`Rut UNIQUE`
`UnidadMedida`	`UQ_UnidadMedida_Codigo`	`Codigo UNIQUE`
`Producto`	`UQ_Producto_Sku`	`Sku UNIQUE`
`Producto`	`CK_Producto_Precios`	`PrecioCompra >= 0 AND (PrecioVenta IS NULL OR PrecioVenta >= 0)`
`Producto`	`CK_Producto_Stock`	`Stock >= 0 AND StockMinimo >= 0`
`Producto`	`DF_Producto_Activo` , `DF_Producto_FechaCreacion`	`Activo` por defecto `1` , `FechaCreacion` por defecto `SYSDATETIME()`
`MovimientoStock`	`CK_Movimiento_Tipo`	`Tipo IN ('ENTRADA','SALIDA','AJUSTE')`
`MovimientoStock`	`CK_Movimiento_Cantidad`	`Cantidad > 0`
`MovimientoStock`	`CK_Movimiento_Stock`	`StockAnterior >= 0 AND StockNuevo >= 0`
`Auditoria`	`CK_Auditoria_Operacion`	`Operacion IN ('INSERT','UPDATE','DELETE','LOGIN','LOGIN_FALLIDO','LOGOUT','ACCESO_DENEGADO','REPORTE_PDF')`

3.3 Índices para el rendimiento (RNF-04)

Índice	Tabla	Columnas	Uso
`IX_Producto_Nombre`	`Producto`	`Nombre`	Búsqueda por texto en el mantenedor
`IX_Producto_Categoria`	`Producto`	`IdCategoria`, `Activo`	Filtro por categoría
`IX_Producto_Proveedor`	`Producto`	`IdProveedor`	Filtro por proveedor
`IX_Producto_Critico`	`Producto`	`Stock`, `StockMinimo` (`INCLUDE`)	Filtro de stock crítico y reporte PDF
`IX_Movimiento_Producto_Fecha`	`MovimientoStock`	`IdProducto`, `Fecha DESC`	Historial de movimientos
`IX_Auditoria_Fecha`	`Auditoria`	`Fecha DESC`	Consulta del log y reporte PDF
`IX_Auditoria_Usuario`	`Auditoria`	`UsuarioAplicacion`, `Fecha DESC`	Filtro por usuario en el log

---

4. Vista de apoyo y decisiones de modelado

- `vw_ProductoDetalle`: vista que resuelve los `JOIN` con `Categoria`, `Proveedor` y `UnidadMedida` y expone `EsStockCritico` y `PorVencer90`; es consumida por los SPs del mantenedor y del reporte de stock crítico, de modo que la aplicación nunca escribe SQL de negocio.
- **`RowVersion` (rowversion)** en `Producto`, `Categoria`, `Proveedor` y `Usuario`: soporta la concurrencia optimista exigida por RN-08; el SP compara el token recibido con el actual y lanza error si cambió.
- **Baja lógica (`Activo`)**: preserva la integridad histórica y permite que el trigger registre la baja como un `UPDATE` (no se pierde información de auditoría).
- **JSON en `Auditoria`** (`DatosAnteriores`, `DatosNuevos`): permite auditar estructuras variables sin crear una tabla por entidad, manteniendo el esquema en 3FN; el PDF del log presenta el resumen legible.
- **`UsuarioAplicacion` en la tabla de auditoría** se obtiene con `SESSION_CONTEXT('UsuarioApp')`, establecido por la capa DAL en cada conexión, garantizando identificar al usuario funcional y no solo la cuenta de base de datos.

AIEP de la Universidad Andrés Bello

PARTE 6 DE 10

Arquitectura de la Aplicación

Diseño en cuatro capas (UI, BLL, DAL, Entidades) y flujo de datos

`Documento 06 del repositorio (docs/06_Arquitectura_4_Capas.md)`

Ficha del documento	
Proyecto	SIGBIM — Sistema de Gestión de Bodega de Insumos Médicos
Asignatura	Desarrollo e Ingeniería de Software (PRO205)
Institución	AIEP — Universidad Andrés Bello
Estudiante	Jonathan Felipe Urbina Monasterio — `JONATHAN.URBINA@correoaiep.cl`
Documento	`docs/06_Arquitectura_4_Capas.md` (Hito 1)
Fecha de emisión	septiembre de 2026

Diseño estructural de la solución, responsabilidades por capa, reglas de dependencia y flujo de datos extremo a extremo, tal como se implementa en `/src`.

1. Vista general

```
┌──────────────────────────────────────────────────────────────────────────┐
│ CAPA DE PRESENTACIÓN (UI)  —  PRO205.UI.WinForms (net8.0-windows)        │
│ FrmLogin · FrmPrincipal · FrmProductos · FrmProductoEdicion · FrmCategorias  │
│ FrmProveedores · FrmMovimientos · FrmUsuarios · FrmAuditoria             │
│ Responsabilidad: capturar datos, validar formato, mostrar resultados y errores│
└──────────────────────────────────────────────────────────────────────────┘
                │ llama a servicios (nunca a DAL ni a SQL)
                ▼
┌──────────────────────────────────────────────────────────────────────────┐
│ CAPA DE LÓGICA DE NEGOCIO (BLL)  —  PRO205.BLL (net8.0)                  │
│ ProductoService · CategoriaService · ProveedorService · MovimientoStockService│
│ SeguridadService · AuditoriaService · ReporteService · Validador        │
│ Responsabilidad: reglas de negocio, validaciones, permisos, orquestación │
└──────────────────────────────────────────────────────────────────────────┘
        │ invoca repositorios                          │ usa entidades
        ▼                                              ▼
┌──────────────────────────────────┐   ┌──────────────────────────────────┐
│ CAPA DE ACCESO A DATOS (DAL) —   │   │ CAPA DE ENTIDADES —              │
│ PRO205.DAL (net8.0)              │   │ PRO205.Entidades (net8.0)        │
│ ConexionSql · ProductoDAL · CategoriaDAL · ProveedorDAL · MovimientoStockDAL │
│ UsuarioDAL · RolDAL · AuditoriaDAL   │   │ POCOs + enums + DTOs de filtro   │
│ Responsabilidad: ejecutar SPs, mapear filas a entidades, traducir excepciones │
└──────────────────────────────────┘   └──────────────────────────────────┘
        │ SqlConnection + CommandType.StoredProcedure
        ▼
┌──────────────────────────────────────────────────────────────────────────┐
│ BASE DE DATOS SQL Server — /database                                     │
│ Tablas normalizadas · SPs de CRUD · Triggers de auditoría · Restricciones │
└──────────────────────────────────────────────────────────────────────────┘
```

Complemento transversal: `PRO205.Reportes` (QuestPDF) genera los documentos PDF a partir de los datos y metadatos que le entrega la BLL.

1.1 Regla de dependencias (unidireccional)

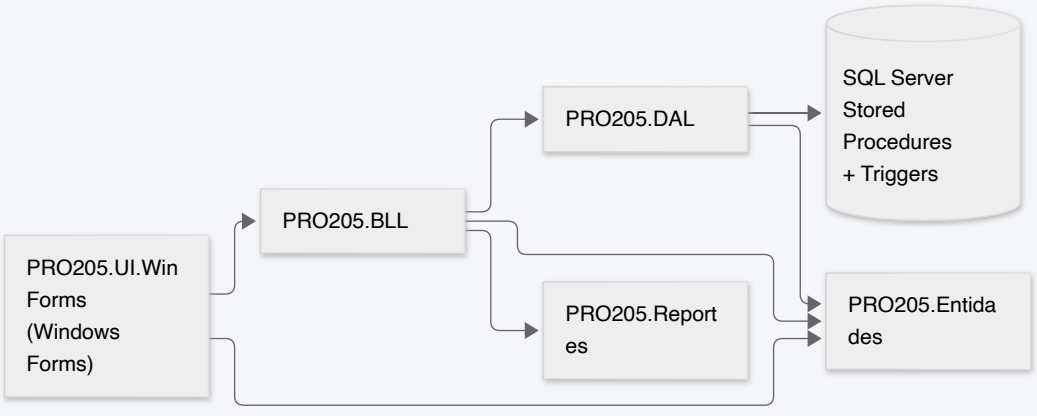


Figura 1 — Diagrama del documento (fuente Mermaid versionada en el repositorio).

No existe referencia de BLL hacia UI, ni de DAL hacia BLL: cualquier dependencia inversa se considera un defecto de arquitectura (criterio 5 de la rúbrica).

2. Responsabilidades por capa

Capa	Proyecto	Responsabilidades	Prohibiciones
Presentación (UI)	`PRO205.UI.WinForms`	Construir formularios, capturar y validar formato (campos obligatorios, tipos, longitudes), invocar servicios de la BLL, presentar resultados y mensajes de error funcionales, habilitar/deshabilitar funciones según permisos	No ejecuta SQL, no conoce `SqlConnection`, no aplica reglas de negocio (más allá de la validación de formato)
Lógica de Negocio (BLL)	`PRO205.BLL`	Validaciones de negocio, reglas de stock, cálculo de criticidad, autenticación y política de contraseña, autorización por permisos, orquestación de reportes, traducción de errores de datos a mensajes funcionales	No conoce controles de Windows Forms, no abre conexiones ni ejecuta comandos
Acceso a Datos (DAL)	`PRO205.DAL`	Crear la conexión, establecer `SESSION_CONTEXT('UsuarioApp')`, invocar los SPs con parámetros tipados, mapear `SqlDataReader` a entidades, capturar `SqlException` y lanzar `DatosException` con código y mensaje funcional	No contiene reglas de negocio ni SQL embebido: solo el nombre del SP y sus parámetros
Entidades	`PRO205.Entidades`	Modelar el dominio (POCO), enumeraciones, DTOs de filtro y resultados (`ResultadoOperacion`)	No tiene dependencias de otras capas ni de librerías de acceso a datos
Reportes	`PRO205.Reportes`	Generar el PDF (encabezado, filtros, tabla, totales, pie con usuario emisor y fecha) y administrar la carpeta de salida	No consulta la base de datos por sí mismo: recibe datos ya resueltos

3. Flujo de datos extremo a extremo

3.1 Crear un insumo (con auditoría automática)

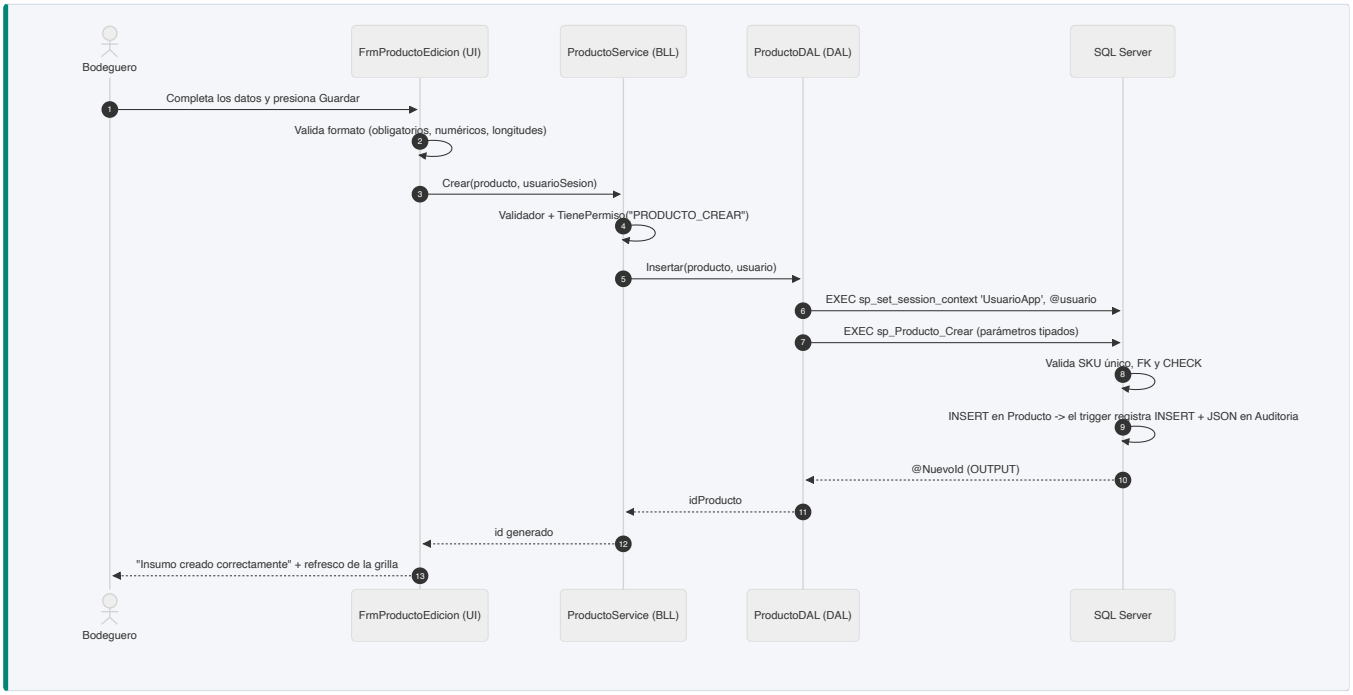


Figura 2 — Diagrama del documento (fuente Mermaid versionada en el repositorio).

3.2 Inicio de sesión con control de intentos

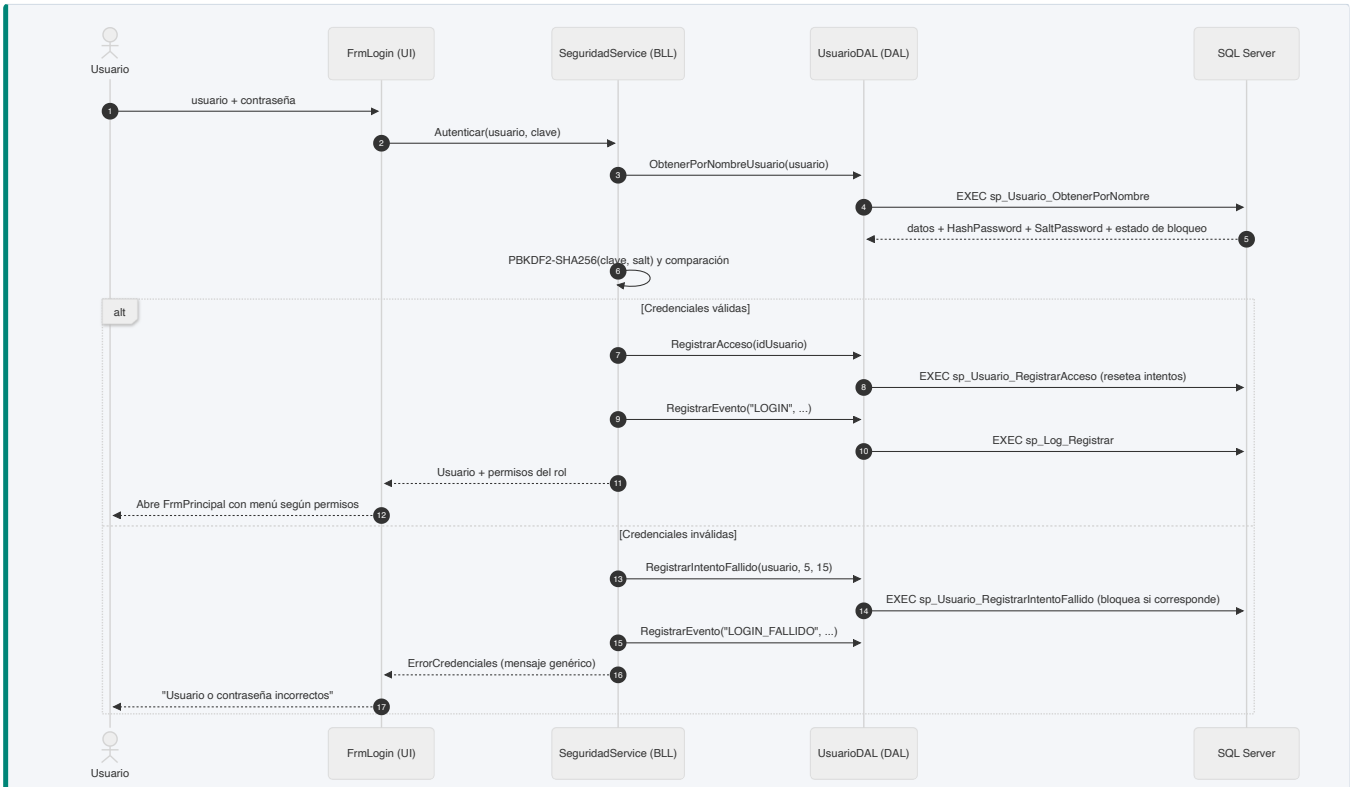


Figura 3 — Diagrama del documento (fuente Mermaid versionada en el repositorio).

3.3 Generar el reporte PDF del log de auditoría

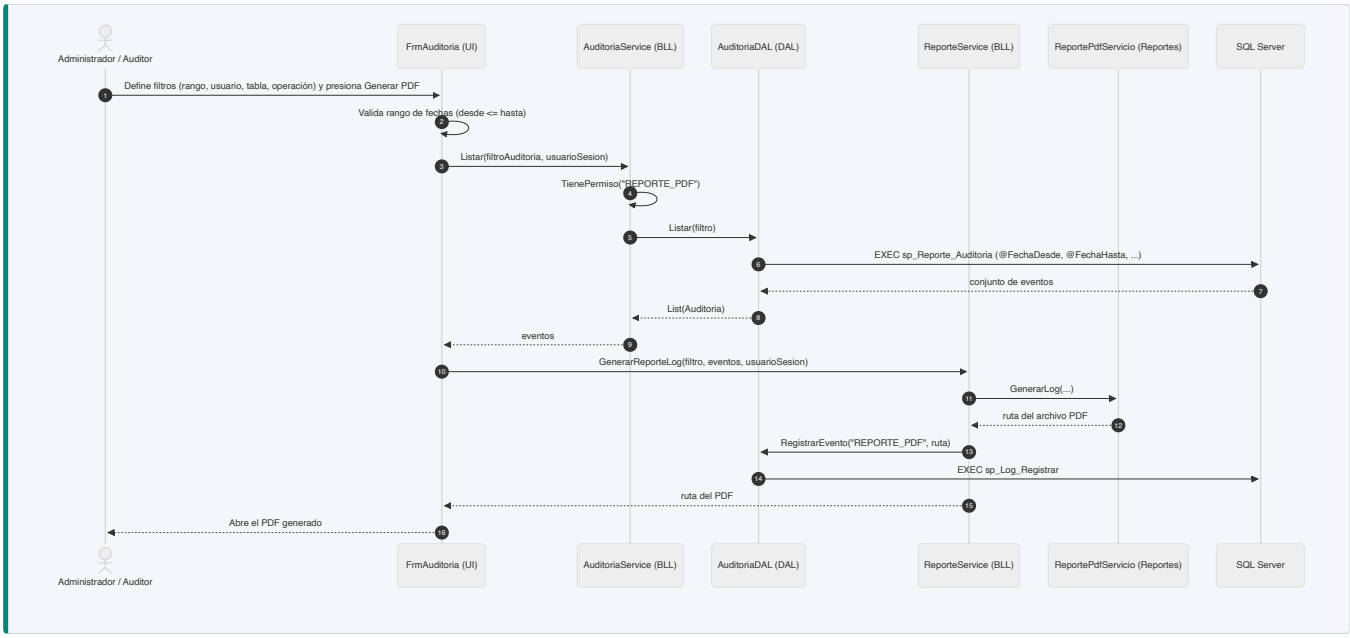


Figura 4 — Diagrama del documento (fuente Mermaid versionada en el repositorio).

3.4 Registrar un movimiento de stock (regla de negocio en la base)

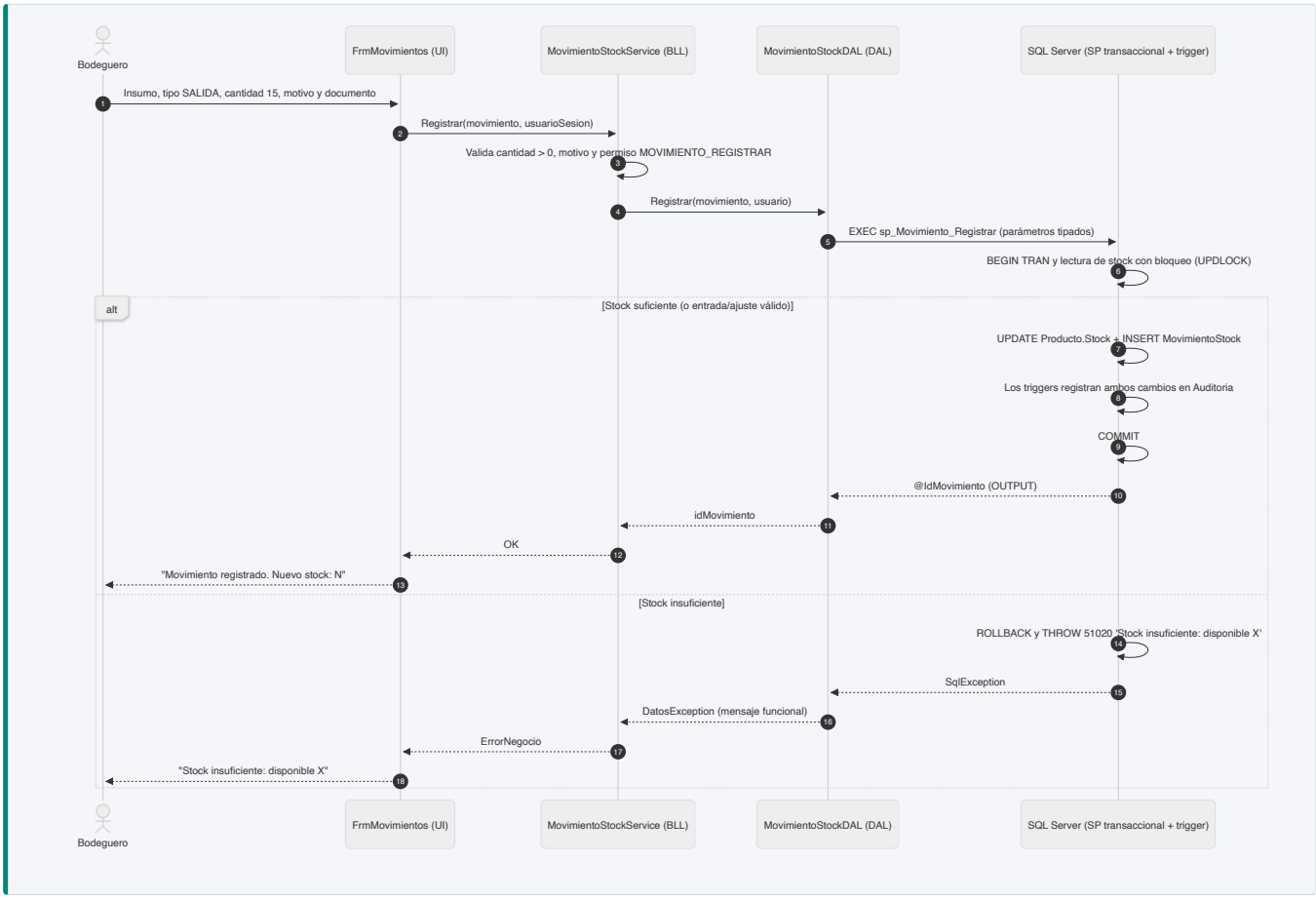


Figura 5 — Diagrama del documento (fuente Mermaid versionada en el repositorio).

4. Validaciones y manejo de errores por capa

Nivel	Qué valida	Ejemplo	Acción ante error
UI (presentación)	Formato y obligatoriedad de los campos, tipos numéricos y rangos de fecha	`Sku` vacío, `PrecioCompra = -100`, fecha "desde" mayor que "hasta"	Marca el control, muestra el mensaje y no invoca la BLL
BLL (negocio)	Reglas del negocio, permisos, política de contraseña y coherencia de estados	Salida mayor al stock, insumo inactivo, contraseña que no cumple la política, rol sin permiso	Lanza `ErrorNegocioException` con mensaje funcional y audita `ACCESO_DENEGADO` cuando corresponde
DAL (acceso a datos)	Traducción de errores del motor y mapeo de resultados	`SqlException` 2627 (`UNIQUE`), 547 (FK/`CHECK`), 51020 (regla de stock)	Convierte a `DatosException` con código y mensaje funcional, sin exponer detalles técnicos (RNF-10)
BD (SQL Server)	Integridad estructural y reglas complejas dentro de transacción	SKU duplicado, FK inexistente, stock negativo, concurrencia (`RowVersion`)	`THROW` con mensaje de negocio y trigger que garantiza la trazabilidad

5. Estructura de la solución y patrones aplicados

```
src/
├─ PRO205.Mantenedor.sln
├─ PRO205.Entidades/          # entidades POCO, enumeraciones, DTOs de filtro y resultados
├─ PRO205.DAL/                # ConexionSql (fábrica de conexión + SESSION_CONTEXT) y repositorios por entidad
├─ PRO205.BLL/                # servicios de negocio, validador, seguridad y sesión de aplicación
├─ PRO205.Reportes/           # generación de PDF (QuestPDF) para el log y el stock crítico
├─ PRO205.UI.WinForms/        # formularios WinForms + appsettings.template.json
└─ tools/PRO205.SmokeTest/    # verificación automatizada de DAL/BLL/Reportes (multiplataforma)
```

Patrón / decisión	Aplicación en el proyecto
Separación en capas (SoC)	Cada capa expone solo lo necesario: la UI no conoce SQL y la BLL no conoce controles de Windows Forms
Repository / Service Layer	Los `*DAL` encapsulan el acceso a los SPs y los `*Service` exponen las operaciones de negocio
DTO	`FiltroProducto`, `FiltroAuditoria` y `ResultadoOperacion` transportan criterios y resultados entre capas

Patrón / decisión	Aplicación en el proyecto
Fábrica de conexión (`ConexionSql`)	Único punto que resuelve la cadena de conexión (archivo o variable de entorno) y establece el usuario de sesión en el motor
Excepciones de dominio (`ErrorNegocioException` , `DatosException`)	Permiten que la UI muestre mensajes funcionales sin filtrar el error técnico (RNF-10)
Concurrencia optimista	Columna `RowVersion` verificada dentro de los SPs de actualización
Auditoría delegada al motor	Triggers independientes del código de la aplicación (A.8.15)

6. Trazabilidad arquitectura ↔ requisitos ↔ rúbrica

Criterio evaluado	Cómo se cumple	Evidencia
Diseño y estructura en capas (4 capas, bajo acoplamiento)	Cinco proyectos con dependencias unidireccionales y responsabilidad única	`docs/06` secciones 1 y 2; diagrama de clases de arquitectura en `docs/04` sección 4; `src/*.csproj`
Mantenedor CRUD invocando los SPs	UI → BLL → DAL → `CommandType.StoredProcedure`	`docs/06` secciones 3 y 5; `src/PRO205.DAL`
Validaciones y manejo de errores SQL	Validación de formato en UI, de negocio en BLL y traducción de `SqlException` en DAL	`docs/06` sección 4; `docs/07` casos de prueba negativos
Modelo de BD normalizado con SPs	Esquema en 3FN + procedimientos almacenados + triggers de auditoría	`docs/05` ; `database/01` a `database/03`
Reporte PDF del log	`ReporteService` + `ReportePdfServicio` sobre `sp_Reporte_Auditoria`	`docs/06` sección 3.3; `src/PRO205.Reportes`

AIEP de la Universidad Andrés Bello

PARTE 7 DE 10

Plan de Pruebas y Evidencias

Estrategia de pruebas, casos de prueba CP-01 a CP-30 y evidencias del sistema

`Documento 07 del repositorio (docs/07_Plan_de_Pruebas_y_Evidencias.md)`

Ficha del documento	
Proyecto	SIGBIM — Sistema de Gestión de Bodega de Insumos Médicos
Asignatura	Desarrollo e Ingeniería de Software (PRO205)
Institución	AIEP — Universidad Andrés Bello
Estudiante	Jonathan Felipe Urbina Monasterio — `JONATHAN.URBINA@correoaiep.cl`
Documento	`docs/07_Plan_de_Pruebas_y_Evidencias.md`
Fecha de emisión	septiembre de 2026

Complementa los requisitos RF/RNF de `docs/02_Requerimientos_RF_RNF_ISO27001.md` y verifica los controles declarados de ISO/IEC 27001:2022.

---

1. Estrategia de pruebas

Nivel	Objetivo	Herramienta	Entorno
N1 — Compilación	Que las 4 capas y la UI compilen sin errores ni advertencias	`dotnet build src/PRO205.Mantenedor.sln`	macOS (verificado) y Windows
N2 — Verificación funcional automatizada	Ejercitar DAL + BLL + Reportes de extremo a extremo sin interfaz	`PRO205.SmokeTest` (`dotnet run --project tools/PRO205.SmokeTest`)	macOS apuntando a SQL Server o Windows
N3 — Reportes PDF	Generar los dos documentos con datos controlados y verificar contenido	`PRO205.SmokeTest -- --demo-pdf`	macOS (verificado) y Windows
N4 — Pruebas manuales de UI	Validar el mantenedor CRUD, la sesión, los permisos y los mensajes	Aplicación WinForms	Windows
N5 — Pruebas negativas (reglas de negocio e integridad)	Confirmar que la base y la BLL **rechazan** datos inválidos	UI + `sqlcmd`	Windows
N6 — Pruebas de seguridad y trazabilidad	Verificar bloqueo por intentos, control de permisos, auditoría y robustez del log	UI + `sqlcmd`	Windows
N7 — Portabilidad	Confirmar que la migración macOS → Windows solo requiere ajustar la conexión	Checklist del documento `docs/08`	Ambos

Criterio de aceptación global: las 7 etapas ejecutadas sin fallos y con evidencia registrada en `docs/evidencias`.

---

2. Casos de prueba

2.1 Pruebas funcionales (mantenedor y operación)

ID	Requisito	Descripción	Datos de entrada	Resultado esperado	Evidencia
CP-01	RF-09	Crear un insumo nuevo	SKU `TEST-001`, nombre "Insumo de prueba", categoría y proveedor válidos, precio compra 1.000, precio venta 1.500, stock mínimo 10	Mensaje de éxito, el insumo aparece en la grilla y existe la fila en `Auditoria` con operación `INSERT`	E-01, E-02
CP-02	RF-09, RN-04	Intento de crear un insumo con SKU duplicado	SKU `TEST-001` (ya existente)	Mensaje "El SKU indicado ya está registrado para otro insumo" y **no** se crea el registro	E-02
CP-03	RF-10	Buscar por texto y filtrar por categoría	Texto "jering", categoría `Insumo desechable`	Grilla solo con los insumos que cumplen ambos criterios	E-01
CP-04	RF-10, RF-11	Filtrar solo stock crítico	Marcar "Solo stock crítico"	Se listan únicamente insumos con `Stock <= StockMinimo` (resaltados)	E-01
CP-05	RF-09	Modificar precio y stock mínimo	Precio compra 1.000 → 1.200	Cambio guardado; en `Auditoria` aparecen los valores anteriores y nuevos en JSON	E-02
CP-06	RF-09, RF-22	Concurrencia optimista	Modificar el mismo insumo desde dos formularios y guardar el segundo	Mensaje "El registro fue modificado por otro usuario, recargue la información" (error 51040)	E-02
CP-07	RF-09, RN-02	Baja lógica de un insumo	Seleccionar insumo y "Dar de baja"	El insumo queda `Activo = 0`, se excluye de la grilla y el cambio queda auditado como `UPDATE (baja lógica)`	E-02
CP-08	RF-07	Baja de categoría con insumos activos	Categoría con 3 insumos activos	Mensaje "No es posible dar de baja la categoría: tiene insumos activos asociados" (error 51004)	E-03
CP-09	RF-08	Validación del RUT del proveedor	RUT `11.111.111-1` (dígito verificador incorrecto)	Mensaje "El RUT ingresado no es válido"; no se invoca la base	E-03

ID	Requisito	Descripción	Datos de entrada	Resultado esperado	Evidencia
CP-10	RF-12	Registro de entrada de stock	Insumo `JER-5ML`, tipo ENTRADA, cantidad 100, motivo "Recepción factura 10245"	Stock anterior 480 → 500; movimiento registrado; ambos cambios auditados	E-04, E-05
CP-11	RF-12, RN-03	Salida mayor al stock disponible	Insumo con 10 unidades, SALIDA de 15	Mensaje "Stock insuficiente: el insumo X tiene 10 unidades disponibles…"; **ningún** cambio en la base (transacción revertida)	E-04
CP-12	RF-12	Ajuste de inventario por vencimiento	Tipo AJUSTE, cantidad 0 para un insumo vencido	El stock queda exactamente en 0 y el motivo explica la merma	E-04
CP-13	RF-13	Historial de movimientos filtrado	Rango de fechas y tipo SALIDA	Se listan solo los movimientos del período y tipo indicados	E-04
CP-14	RF-19	Reporte PDF de stock crítico	Presionar "PDF stock crítico"	Archivo PDF generado con SKU, stock, mínimo, reposición sugerida y costo estimado	E-06

2.2 Pruebas de seguridad y trazabilidad

ID	Requisito	Descripción	Resultado esperado	Evidencia
CP-15	RF-01	Inicio de sesión con credenciales correctas	Se abre el panel principal con nombre y rol; evento `LOGIN` en `Auditoria` y `UltimoAcceso` actualizado	E-01
CP-16	RF-01, RF-16	Credenciales incorrectas	Mensaje genérico "Usuario o contraseña incorrectos" y evento `LOGIN_FALLIDO` con intentos restantes	E-05
CP-17	RF-02	Cinco intentos fallidos consecutivos	Cuenta bloqueada 15 minutos (`BloqueadoHasta` futuro); el siguiente intento no autentica	E-05
CP-18	RF-03, RF-06	Contraseña que no cumple la política (`abc123`)	Mensaje con los requisitos (largo, mayúscula, minúscula, dígito, símbolo); no se guarda	E-07
CP-19	RF-03	Usuario `auditor` (rol Consulta) intenta abrir Usuarios	La opción aparece deshabilitada; forzada la operación, la BLL la rechaza y registra `ACCESO_DENEGADO`	E-05
CP-20	RF-15	Modificación del precio de un insumo	Fila en `Auditoria` con JSON de valores anteriores y nuevos y usuario correcto	E-02
CP-21	RF-15, RNF-09	Cambio de contraseña de un usuario	En el log, `HashPassword` y `SaltPassword` aparecen como `***`	E-07
CP-22	RN-06	`UPDATE dbo.Auditoria SET Detalle='alterado' WHERE IdAuditoria=1;`	Error 51030 ("registros inmutables") y la operación se revierte	E-05
CP-23	RF-18	Generar el PDF del log con filtros	PDF con filtros, emisor, totales y tabla de eventos; queda el evento `REPORTE_PDF`	E-06
CP-24	RF-20	Presionar "Cerrar sesión"	Evento `LOGOUT` auditado y retorno al inicio de sesión	E-05

2.3 Pruebas de validación de datos (negativas)

ID	Requisito	Entrada	Resultado esperado
CP-25	RF-14	Precio de compra `-100`	El control numérico impide valores negativos; no permite guardar
CP-26	RF-14	Nombre `"ab"`	"El nombre del insumo es obligatorio (mínimo 3 caracteres)"
CP-27	RF-14	Compra 2.000, venta 1.500	"El precio de venta no puede ser menor que el precio de compra" (BLL y SP)
CP-28	RF-14	Correo `usuario.sincorreo`	"El correo electrónico no tiene un formato válido"
CP-29	RF-23	`sp_Producto_Crear` con `@IdCategoria = 9999`	Error 51003: "La categoría seleccionada no existe o está inactiva"
CP-30	RF-24, RNF-10	Cadena de conexión a un servidor inexistente	Mensaje funcional sin trazas técnicas; la aplicación permanece operativa

3. Matriz de cobertura requisito ↔ caso de prueba

Requisito	Casos de prueba	Estado en macOS	Estado en Windows
RF-01, RF-02 (autenticación y bloqueo)	CP-15, CP-16, CP-17	Pendiente (requiere SQL Server)	Pendiente
RF-03, RF-05 (autorización por rol)	CP-19	Verificado por código (BLL)	Pendiente
RF-04, RF-06 (usuarios y contraseña)	CP-18, CP-21	Pendiente (requiere SQL Server)	Pendiente
RF-07, RF-08 (categorías y proveedores)	CP-08, CP-09	Verificado por código	Pendiente
RF-09, RF-10, RF-11 (mantenedor de insumos)	CP-01 a CP-07	Verificado por compilación	Pendiente

Requisito	Casos de prueba	Estado en macOS	Estado en Windows
RF-12, RF-13 (movimientos de stock)	CP-10 a CP-13	Pendiente (requiere SQL Server)	Pendiente
RF-14 (validaciones)	CP-25 a CP-28	Verificado por código	Pendiente
RF-15, RF-16, RF-17 (auditoría)	CP-20 a CP-22	Verificado por script SQL	Pendiente
RF-18, RF-19 (reportes PDF)	CP-14, CP-23	**Verificado: PDF generados (E-06)**	Pendiente
RF-20 (cierre de sesión)	CP-24	Verificado por código	Pendiente
RF-21 a RF-25 (persistencia e integridad)	CP-29, CP-30	Verificado por smoke test	Pendiente

4. Evidencias

Las evidencias se almacenan en `docs/evidencias/`. Las capturas de pantalla se agregan al ejecutar las pruebas en el equipo Windows.

Código	Evidencia	Estado
E-01	Captura del inicio de sesión y del panel principal con la grilla de insumos filtrada	Pendiente (Windows)
E-02	Captura del evento `UPDATE` en el log con el detalle JSON de valores anteriores/nuevos	Pendiente (Windows)
E-03	Captura de los mensajes de validación (RUT inválido, categoría con insumos activos)	Pendiente (Windows)
E-04	Captura del registro de un movimiento de stock y su efecto sobre el stock	Pendiente (Windows)
E-05	Captura del log con eventos de seguridad y de la prueba de inmutabilidad	Pendiente (Windows)
E-06	**PDF de muestra del log de auditoría y del stock crítico**	✅ `docs/evidencias/Reporte_Auditoria_PDF_muestra.pdf` · `Reporte_StockCritico_PDF_muestra.pdf`
E-07	Captura del cambio de contraseña y del enmascaramiento (`***`) en el log	Pendiente (Windows)
E-08	Captura de `scripts/verificar_entorno.ps1` y del respaldo de la base	Pendiente (Windows)

4.1 Registro de ejecución

Fecha	Entorno	Actividad	Resultado
Sep-2026	macOS (arm64, SDK .NET 10)	`dotnet build src/PRO205.Mantenedor.sln` (incluye la UI con `EnableWindowsTargeting`)	✅ Compilación correcta: 0 advertencias, 0 errores
Sep-2026	macOS	`dotnet run --project tools/PRO205.SmokeTest` sin SQL Server local	✅ Detecta la ausencia del servidor y muestra el mensaje funcional esperado (CP-30)
Sep-2026	macOS	`dotnet run --project tools/PRO205.SmokeTest -- --demo-pdf`	✅ Genera los dos reportes PDF (E-06)
Pendiente	Windows 10/11 + SQL Server	Scripts `database/01` a `04` y pruebas CP-01 a CP-30	—

5. Trazabilidad con los controles ISO/IEC 27001:2022

Control	Verificación en las pruebas
A.5.15 / A.8.2 / A.8.3	CP-19 (permisos por rol y funciones deshabilitadas)
A.5.16 / A.5.18	CP-01, CP-18, CP-19 (alta de usuarios y asignación de roles)
A.5.17 / A.8.24	CP-18, CP-21 (política y derivación de credenciales)
A.8.5	CP-16, CP-17 (intentos fallidos y bloqueo temporal)
A.8.9	CP-30 (configuración de conexión externalizada)
A.8.11 / A.8.12	CP-21 (enmascaramiento de credenciales en el log)
A.8.13	E-08 (respaldo y procedimiento de restauración)
A.8.15 / A.8.16	CP-20, CP-22, CP-23 (registro, inmutabilidad y supervisión)
A.8.28	CP-02, CP-11, CP-29 (validaciones en la base y ausencia de SQL embebido)

PARTE 8 DE 10

Manual de Ejecución en Windows

Instalación, puesta en marcha, respaldo y portabilidad macOS → Windows

Documento 08 del repositorio (docs/08_Manual_Ejecucion_Windows.md)

Ficha del documento	
Proyecto	SIGBIM — Sistema de Gestión de Bodega de Insumos Médicos
Asignatura	Desarrollo e Ingeniería de Software (PRO205)
Institución	AIEP — Universidad Andrés Bello
Estudiante	Jonathan Felipe Urbina Monasterio — `JONATHAN.URBINA@correoaiep.cl`
Documento	`docs/08_Manual_Ejecucion_Windows.md`
Fecha de emisión	septiembre de 2026

Guía de instalación, puesta en marcha y verificación del sistema en el **equipo Windows de entrega** (SQL Server + aplicación WinForms). Corresponde al proceso de paso de macOS → Windows exigido por la asignatura.

---

1. Requisitos del equipo

Componente	Versión mínima	Para qué se usa
Windows 10 / 11 (64 bits)	—	Sistema operativo de la aplicación WinForms
.NET SDK	8.0 o superior	Compilar la solución (`dotnet build`)
.NET Desktop Runtime	8.0 (incluido en el SDK)	Ejecutar la interfaz Windows Forms
SQL Server	Express / Developer / LocalDB 2019+	Base de datos `SigbimDb`
SQL Server Command Line Utilities	—	Ejecutar los scripts con `sqlcmd` (alternativa: SSMS o Azure Data Studio)
Git	—	Clonar el repositorio y registrar el avance

---

2. Instalación del entorno (una sola vez)

2.1 .NET SDK

```
winget install Microsoft.DotNet.SDK.8
# Verificar
dotnet --list-sdks
dotnet --list-runtimes | Select-String WindowsDesktop
```

2.2 SQL Server Express (incluye LocalDB)

```
winget install Microsoft.SQLServer.2022.Express
winget install Microsoft.SQLServerManagementStudio   # opcional pero recomendado
```

Habilite el **protocolo TCP/IP** (SQL Server Configuration Manager → Configuración de red de SQL Server → Protocolos de SQLEXPRESS → TCP/IP = Habilitado) y reinicie el servicio `MSSQL$SQLEXPRESS` . Habilite también el **modo mixto** de autenticación si usará un usuario de aplicación.

2.3 Herramientas de línea de comandos

```
winget install Microsoft.Sqlcmd
sqlcmd -?    # comprobar
```

2.4 Git y obtención del proyecto

```
git clone <URL-del-repositorio> PRO205-JONATHAN.URBINA
cd PRO205-JONATHAN.URBINA
```

> Si recibe el proyecto como archivo comprimido, descomprímalo en una ruta **sin espacios ni acentos** (por ejemplo `C:\PRO205\SIGBIM`).

---

3. Creación de la base de datos

3.1 Ejecución de los scripts (orden estricto)

```
cd database
sqlcmd -S localhost\SQLEXPRESS -E -i 01_esquema.sql
sqlcmd -S localhost\SQLEXPRESS -E -i 02_stored_procedures.sql
sqlcmd -S localhost\SQLEXPRESS -E -i 03_triggers_auditoria.sql
sqlcmd -S localhost\SQLEXPRESS -E -i 04_seed_datos.sql
```

> Si instaló SQL Server Developer/Standard (instancia por omisión), use `-S localhost` o `-S .`. Con SSMS/Azure Data Studio: abra cada archivo y ejecute el script completo en el mismo orden.

3.2 Verificación de la creación

```
sqlcmd -S localhost\SQLEXPRESS -E -d SigbimDb -Q "SELECT COUNT(*) AS Sps FROM sys.procedures WHERE schema_id = SCHEMA_ID('dbo');"
sqlcmd -S localhost\SQLEXPRESS -E -d SigbimDb -Q "SELECT COUNT(*) AS Triggers FROM sys.triggers WHERE parent_class = 1;"
sqlcmd -S localhost\SQLEXPRESS -E -d SigbimDb -Q "SELECT COUNT(*) AS Insumos FROM dbo.Producto; SELECT COUNT(*) AS EventosAuditoria FROM dbo.Auditoria;"
sqlcmd -S localhost\SQLEXPRESS -E -d SigbimDb -Q "SELECT Sku, Nombre, Stock, StockMinimo FROM dbo.vw_ProductoDetalle WHERE EsStockCritico = 1;"
```

Resultados esperados: **37 procedimientos almacenados**, **7 triggers**, **12 insumos** cargados por la semilla y varios eventos de auditoría generados automáticamente por los triggers al insertar la semilla.

3.3 Prueba de trazabilidad (opcional pero recomendada)

```
EXEC sp_set_session_context N'UsuarioApp', N'prueba.trazabilidad';
UPDATE dbo.Producto SET PrecioCompra = PrecioCompra + 10 WHERE Sku = 'JER-5ML';

SELECT TOP 1 Operacion, Tabla, RegistroId, UsuarioAplicacion, Fecha, DatosAnteriores, DatosNuevos, Detalle
FROM   dbo.Auditoria WHERE Tabla = 'Producto' ORDER BY IdAuditoria DESC;

-- Debe fallar (RN-06: el log es inmutable)
UPDATE dbo.Auditoria SET Detalle = 'alterado' WHERE IdAuditoria = 1;
```

---

4. Configuración de la conexión (fuera del código fuente)

El sistema lee la cadena de conexión en este orden: **variable de entorno** `SIGBIM_CONN` → `appsettings.json` → valor por omisión de laboratorio (`localhost` con autenticación integrada).

Opción A — archivo `appsettings.json` (recomendada)

```
cd ..\src\PRO205.UI.WinForms
copy appsettings.template.json appsettings.json
notepad appsettings.json
```

```
{
  "ConnectionStrings": {
    "Sigbim": "Server=localhost\\SQLEXPRESS;Database=SigbimDb;Integrated Security=True;TrustServerCertificate=True;Encrypt=False;"
  }
}
```

Opción B — variable de entorno (sin dejar archivo en el equipo)

```
setx SIGBIM_CONN "Server=localhost\SQLEXPRESS;Database=SigbimDb;Integrated Security=True;TrustServerCertificate=True;Encrypt=False;"
# Abra una consola nueva para que la variable quede disponible
```

> Control **A.8.9/A.8.24**: `appsettings.json` está en `.gitignore`, por lo que **nunca** se versiona; solo la plantilla `appsettings.template.json` viaja en el repositorio.

---

5. Compilación y ejecución

```
cd src
dotnet restore
dotnet build
dotnet run --project PRO205.UI.WinForms
```

5.1 Primer inicio de sesión

Usuario	Contraseña inicial	Rol
`admin`	`Admin#2026`	Administrador
`bod.central`	`Bodega#2026`	Bodeguero
`auditor`	`Auditor#2026`	Consulta (solo lectura)

Todos tienen `DebeCambiarPassword = 1`: el sistema **exige** cambiar la contraseña en el primer acceso (política: mínimo 10 caracteres con mayúscula, minúscula, dígito y símbolo).

5.2 Recorrido funcional sugerido (demostración / evaluación)

1. **Inicie sesión** como `admin` → cambie la contraseña cuando el sistema lo solicite.

2. **Usuarios y roles**: revise la grilla y los permisos del rol seleccionado en el panel derecho.
3. **Categorías** y **Proveedores**: cree, modifique y dé de baja un registro de prueba.
4. **Insumos médicos** (entidad principal): cree un insumo (SKU único), modifique precio y stock mínimo, filtre por texto/categoría/stock crítico y dé de baja un insumo.
5. **Movimientos de stock**: registre una ENTRADA (recepción) y una SALIDA (entrega a box); intente una salida mayor al stock para ver el rechazo de la regla RN-03.
6. **Log de auditoría**: consulte los eventos generados, abra "Ver detalle JSON" para ver valores anteriores y nuevos, y presione **Generar reporte PDF**.
7. **Autentíquese como** `auditor`: verifique que los módulos de escritura aparecen deshabilitados y que un intento de acceso indebido queda registrado como `ACCESO_DENEGADO`.

5.3 Verificación automatizada (opcional pero muy Recomendada)

```
# Comprobación del equipo completo (SQL Server, .NET, conexión, SPs, triggers)
powershell -ExecutionPolicy Bypass -File ..\scripts\verificar_entorno.ps1

# Verificación funcional de DAL, BLL y reportes PDF contra la base real
dotnet run --project tools\PRO205.SmokeTest

# Generar solo los PDF de muestra (sin base de datos)
dotnet run --project tools\PRO205.SmokeTest -- --demo-pdf
```

Los reportes PDF se guardan en `Documentos\SIGBIM\Reportes` (o en `src\tools\PRO205.SmokeTest\reportes_pdf` cuando se usa el modo demo).

---

6. Respaldo de la base de datos

```
mkdir C:\SIGBIM\Backup
sqlcmd -S localhost\SQLEXPRESS -E -i ..\scripts\respaldo_bd.sql
```

El script genera el respaldo completo (`SigbimDb_completo.bak`), el respaldo del log (`SigbimDb_log.trn`), verifica la legibilidad del archivo y lista el historial de respaldos. Las sentencias de **restauración** están incluidas al final del script, comentadas.

---

7. Diferencias entre macOS (desarrollo) y Windows (entrega)

Aspecto	macOS	Windows
Interfaz Windows Forms	Se **compila** (`EnableWindowsTargeting=true`), no se ejecuta	Se compila y se ejecuta
Base de datos	No hay SQL Server local; se prueba con `--demo-pdf` y mensajes de error controlados	SQL Server Express/LocalDB con la base creada
Propósito de la prueba	`dotnet build` de toda la solución + generación de PDF	Pruebas funcionales CP-01 a CP-30 y evidencias
Ajuste requerido al portar	—	Solo `appsettings.json` (o `SIGBIM_CONN`): **no se modifica código**

Checklist de portabilidad

- [ ] `dotnet build src\PRO205.Mantenedor.sln` sin errores en Windows.
- [ ] Base creada con los 4 scripts y verificación de 37 SPs y 7 triggers.
- [ ] `appsettings.json` creado con el servidor correcto (y sin versionar).
- [ ] Aplicación inicia, autentica y muestra el panel principal.
- [ ] CRUD del insumo probado (crear, modificar, consultar, dar de baja).
- [ ] Movimiento de stock registrado y regla de stock negativo rechazada.
- [ ] Log con eventos y PDF generado.
- [ ] Evidencias capturadas en `docs/evidencias/`.

---

8. Solución de problemas frecuentes

Mensaje / síntoma	Causa probable	Solución
"No fue posible conectar con el servidor SQL Server…"	Servicio detenido, nombre de instancia incorrecto o TCP/IP deshabilitado	Inicie el servicio (`net start MSSQLSERVER` o `MSSQL$SQLEXPRESS`) y corrija `Server=` en `appsettings.json`
"No fue posible abrir la base de datos con las credenciales configuradas"	La base `SigbimDb` no existe o la autenticación integrada no tiene permisos	Ejecute `01_esquema.sql` y conceda permisos al usuario (`05_usuarios_y_permisos_bd.sql`)
"No se encontró el procedimiento almacenado requerido…"	Falta ejecutar `02_stored_procedures.sql`	Ejecute los scripts en el orden 01 → 02 → 03 → 04

Mensaje / síntoma	Causa probable	Solución
"El registro ya existe: se intentó grabar un valor que debe ser único"	SKU, RUT, correo o nombre duplicado	Corrija el dato (la validación es intencional)
"El registro fue modificado por otro usuario…"	Concurrencia optimista (`RowVersion`)	Recargue la grilla y repita la operación
"Stock insuficiente…"	Salida mayor a las existencias (RN-03)	Registre primero una entrada o ajuste la cantidad
"Los registros de auditoría son inmutables…"	Se intentó modificar o borrar el log (RN-06)	Es el comportamiento esperado; solo se puede deshabilitar el trigger para mantenimiento controlado
La aplicación no abre o falla al iniciar	Falta el .NET Desktop Runtime o no hay sesión iniciada	Instale el Desktop Runtime 8.0 y ejecute desde `dotnet run` para ver el detalle
El PDF no se abre automáticamente	No hay aplicación asociada a PDF	Abra el archivo desde la carpeta de reportes indicada en el mensaje

9. Resumen del proceso macOS → Windows

Elemento	Acción realizada
Código fuente	Se escribió una sola vez, sin dependencias de macOS: C# estándar y bibliotecas multiplataforma
Interfaz de usuario	Formularios construidos **en código** (no con el diseñador gráfico), por lo que se comportan idénticos en ambos sistemas
Compilación cruzada	`<EnableWindowsTargeting>true</EnableWindowsTargeting>` en el proyecto de UI para compilar (no ejecutar) en macOS
Conexión a datos	Externalizada en `appsettings.json` / `SIGBIM_CONN` : pasa de un equipo a otro sin recompilar el código
Verificación	En macOS: compilación completa + `PRO205.SmokeTest --demo-pdf` . En Windows: scripts SQL, pruebas CP-01 a CP-30 y evidencias
Archivos que **no** se versionan	`appsettings.json` , `bin/` , `obj/` , respaldos `.bak/.trn` , carpeta `reportes_pdf/`

> El port a Windows **no requiere cambios de código**: basta crear la base de datos, ajustar la cadena de conexión y ejecutar `dotnet run --project PRO205.UI.WinForms` .

AIEP
de la Universidad Andrés Bello

PARTE 9 DE 10

Guía Paso a Paso y Justificación

Construcción del proyecto, decisiones técnicas y trazabilidad con la rúbrica

Documento 09 del repositorio (docs/09_Guia_Paso_a_Paso_y_Justificacion.md)

Ficha del documento	
Proyecto	SIGBIM — Sistema de Gestión de Bodega de Insumos Médicos
Asignatura	Desarrollo e Ingeniería de Software (PRO205)
Institución	AIEP — Universidad Andrés Bello
Estudiante	Jonathan Felipe Urbina Monasterio — `JONATHAN.URBINA@correoaiep.cl`
Documento	`docs/09_Guia_Paso_a_Paso_y_Justificacion.md`
Fecha de emisión	septiembre de 2026

Este documento explica **qué se construyó, en qué orden, cómo se ejecuta y por qué se tomó cada decisión técnica**, relacionando cada decisión con los criterios de la rúbrica y con los controles de ISO/IEC 27001:2022.

---

1. Mapa del repositorio (dónde está cada cosa)

```
PRO205-JONATHAN.URBINA/
├─ README.md                                  Índice general, stack y puesta en marcha
├─ docs/
│  ├─ 01_Informe_Tecnico.md                   Problemática real, objetivos, actores, ISO 27001
│  ├─ 02_Requerimientos_RF_RNF_ISO27001.md    25 RF con tríada, 12 RNF y matriz de controles
│  ├─ 03_Casos_de_Uso.md                      Diagrama UML de casos de uso + especificación textual
│  ├─ 04_Diagrama_de_Clases.md                Diagrama de clases + mapa a las capas
│  ├─ 05_Modelo_Relacional_Normalizacion.md   Normalización 1FN→3FN, integridad referencial e índices
│  ├─ 06_Arquitectura_4_Capas.md              UI/BLL/DAL/Entidades, flujo de datos y patrones
│  ├─ 07_Plan_de_Pruebas_y_Evidencias.md      30 casos de prueba, evidencias y registro de ejecución
│  ├─ 08_Manual_Ejecucion_Windows.md          Instalación, ejecución, respaldo y portabilidad
│  ├─ 09_Guia_Paso_a_Paso_y_Justificacion.md  Este documento
│  ├─ evidencias/                             PDF de muestra generados por el sistema
│  └─ pdf/                                    Versión PDF con acabado profesional del Hito 1
│                                             (6 documentos + informe consolidado)
├─ database/                                  T-SQL: 01 esquema · 02 SPs · 03 triggers · 04 semilla · 05 hardening
├─ scripts/                                   Respaldo de base, verificación del entorno y generación de PDF
├─ tools/documentos/                          Generador Markdown → PDF (Node.js + Chrome headless)
└─ src/
   ├─ PRO205.Mantenedor.sln
   ├─ PRO205.Entidades/   15 clases   → dominio, filtros, excepciones y resultados
   ├─ PRO205.DAL/         10 clases   → fábrica de conexión, repositorios y traducción de errores
   ├─ PRO205.BLL/         10 clases   → servicios de negocio, seguridad PBKDF2 y sesión
   ├─ PRO205.Reportes/     1 clase    → generación de los PDF (QuestPDF)
   ├─ PRO205.UI.WinForms/ 13 archivos → login, panel, mantenedores, movimientos, usuarios y auditoría
   └─ tools/PRO205.SmokeTest/          Verificación funcional y generador de PDF de muestra
```

---

2. Paso a paso de construcción (en el orden en que se realizó)

Fase	Paso	Qué se hizo	Por qué se hizo así (justificación)	Criterio evaluado
0	0.1	Definir la problemática real: bodega de insumos médicos administrada con planillas Excel compartidas	Es un caso **real y aplicable** (contexto laboral) con dolores comprobables: sin control de acceso, sin trazabilidad, sin alertas de stock ni respaldo	Problemática y contexto
0	0.2	Delimitar el alcance: mantenedor de insumos + seguridad + trazabilidad + reportes PDF	Permite dimensionar el trabajo del hito y declarar explícitamente lo que queda fuera (facturación, lotes, integración con ERP)	Delimitación
0	0.3	Elegir la entidad principal `Producto` (insumo) con maestros `Categoria`, `Proveedor`, `UnidadMedida`	La entidad debe tener reglas de negocio verificables (stock, precios, vencimiento) y maestros que permitan demostrar normalización	Alcance funcional
1	1.1	Especificar **25 requisitos funcionales** con la tríada RoI → Necesidad → Criterio DADO/CUANDO/ENTONCES	La rúbrica exige la tríada en todos los RF y criterios verificables	Requerimientos
1	1.2	Especificar **12 requisitos no funcionales** (seguridad, rendimiento, usabilidad, respaldo, portabilidad)	Los atributos de calidad no se expresan como funciones pero condicionan el diseño	RNF
1	1.3	Construir la matriz **ISO/IEC 27001:2022** con 15 controles del Anexo A	Integra controles de seguridad explícitos y trazables a requisitos y código	ISO 27001
1	1.4	Modelar **casos de uso UML** (15 CU con include/extend justificados) y **diagrama de clases** (12 clases)	Evita ambigüedad antes de codificar; los diagramas se escribieron en Mermaid para que GitHub los renderice y queden versionados	UML
1	1.5	Normalizar el modelo de datos partiendo de la planilla Excel real hasta **3FN**	Se documenta el proceso (qué se repetía y cómo se resolvió), no solo el resultado final	Normalización
1	1.6	Diseñar la **arquitectura en 4 capas** con regla de dependencia unidireccional y flujos de datos	Evita código monolítico y acoplamiento entre presentación y datos	Arquitectura
2	2.1	`01_esquema.sql` : 10 tablas con PK/FK/UNIQUE/CHECK/DEFAULT, 8 índices y la vista `vw_ProductoDetalle`	La integridad se garantiza **en el motor**, no solo en la aplicación (defensa en profundidad)	BD e integridad

Fase	Paso	Qué se hizo	Por qué se hizo así (justificación)	Criterio evaluado
2	2.2	`02_stored_procedures.sql` : **37 SPs** de CRUD, movimientos, seguridad, log y reportes	Toda la aplicación opera por SPs parametrizados: elimina SQL embebido (mitiga inyección) y concentra las reglas	SPs
2	2.3	`03_triggers_auditoria.sql` : **7 triggers** AFTER INSERT/UPDATE/DELETE con JSON de valores anteriores/nuevos, enmascaramiento de credenciales y log inmutable	La trazabilidad no depende de que el desarrollador recuerde registrar el cambio	Trazabilidad
2	2.4	`04_seed_datos.sql` : roles, permisos, 3 usuarios con credenciales derivadas y 12 insumos con movimientos iniciales registrados por SP	Permite evaluar con datos realistas y reproducibles, respetando las reglas de negocio en la propia carga	Semilla
3	3.1	Capa **Entidades**: POCOs, enums, filtros y excepciones de dominio	Transporta datos entre capas y encapsula validaciones básicas sin dependencias externas	Capas
3	3.2	Capa **DAL**: `ConexionSql` (cadena externalizada + `SESSION_CONTEXT`), `RepositorioBase` (solo SPs) y repositorios por entidad	Único punto que habla con la base y traduce `SqlException` a mensajes funcionales	Capas y errores
3	3.3	Capa **BLL**: servicios con reglas de negocio, autorización por permisos y política de contraseña **PBKDF2-SHA256 (100.000 iteraciones + sal)**	Concentra las reglas y vuelve a validar lo que la UI podría omitir	Capas y seguridad
3	3.4	Proyecto **Reportes**: `ReportePdfServicio` con QuestPDF para el log de auditoría y el stock crítico	Entrega el módulo PDF exigido y audita la propia generación del reporte	Reportes PDF
3	3.5	Capa **UI WinForms**: 13 archivos (login, panel principal, mantenedores, movimientos, usuarios, auditoría) construidos **en código**	Permite compilar la UI en macOS y ejecutarla en Windows sin cambios ni diseñador gráfico	Mantenedor CRUD
4	4.1	`PRO205.SmokeTest` : verificación funcional de DAL/BLL/Reportes y modo `--demo-pdf`	Permite probar sin interfaz gráfica y generar evidencia de los reportes	Pruebas
4	4.2	`docs/07` (30 casos de prueba), `docs/08` (manual Windows), `scripts/` (respaldo y verificación del entorno)	Cierra los entregables de pruebas, portabilidad y respaldo	Pruebas y respaldo
4	4.3	Versionado incremental en Git con mensajes descriptivos por unidad lógica	Evidencia el avance evolutivo exigido en la rúbrica	Control de versiones
5	5.1	Construir `tools/documentos` (Node.js + Chrome headless) que convierte los Markdown de `/docs` en PDF y genera `docs/pdf/` (6 documentos + informe consolidado)	La entrega se evalúa en PDF: el generador mantiene el Markdown como fuente única y evita el trabajo manual de maquetación	Entrega en PDF

3. Justificación de las decisiones técnicas

#	Decisión	Alternativa descartada	Justificación (por qué)	Respaldo
D-01	**SQL Server con T-SQL**	MySQL / PostgreSQL / archivo JSON	Es el motor exigido por la guía del trabajo; aporta SPs, triggers `AFTER` , `SESSION_CONTEXT` , `ROWVERSION` y `FOR JSON PATH` , todo lo que el diseño necesita	Rúbrica (BD y SPs)
D-02	**Seguridad y trazabilidad dentro del motor** (SPs, restricciones y triggers)	Dejar toda la lógica en C#	Si la lógica vive solo en la aplicación, un cambio de programa o un error de código rompe el control. En el motor, el control es obligatorio e independiente del cliente	A.8.15, A.8.28
D-03	**Todo el acceso a datos por procedimientos almacenados** (37 SPs, `CommandType.StoredProcedure` , parámetros tipados)	SQL embebido o concatenado en C#	Elimina la superficie de inyección SQL, centraliza las reglas y deja la política de datos en el motor (mínimo privilegio)	A.8.28, OWASP A03, RF-21
D-04	**Arquitectura en 4 capas** (UI, BLL, DAL, Entidades) + proyecto de Reportes	Formularios que consultan la base directamente	Cumple el criterio de arquitectura: la UI no conoce SQL, la BLL concentra reglas y el DAL es el único punto de acceso; el sistema es mantenible y las pruebas aislables	Criterio 5 de la rúbrica
D-05	**Contraseñas con PBKDF2-SHA256**, 100.000 iteraciones, sal de 16 bytes por usuario y comparación en tiempo constante	Texto plano, MD5/SHA-1 o SHA-256 simple	Los hashes rápidos son vulnerables a diccionario/GPU; PBKDF2 con sal única por usuario y alto costo computacional es la recomendación vigente, y la comparación en tiempo constante evita filtraciones por tiempo	A.5.17, A.8.24
D-06	**RBAC con tablas** `Rol` , `Permiso` , `RolPermiso`	Roles fijos en el código (`if rol == "admin"`)	Permite cambiar permisos sin recompilar, mantener el principio de mínimo privilegio y auditar quién tiene qué	A.5.15, A.5.18, A.8.2
D-07	**Bloqueo temporal a los 5 intentos** (15 minutos) y **mensaje genérico** de error de credenciales	Mensaje que distinga "usuario no existe" de "contraseña incorrecta"	Mitiga ataques de fuerza bruta y la enumeración de cuentas (OWASP A07); el usuario legítimo recibe un mensaje claro sin pistas al atacante	A.8.5, A.8.16
D-08	**Baja lógica** (`Activo = 0`) en los maestros	`DELETE` físico	Conserva la integridad referencial con movimientos históricos, permite consultar "quién dio de baja" y evita perder trazabilidad	RN-02, A.8.15
D-09	**Auditoría por triggers** con JSON de valores anteriores/nuevos, usuario funcional vía `SESSION_CONTEXT` y enmascaramiento de credenciales	Registrar los cambios desde la aplicación	El registro no depende de la disciplina del programador; el usuario funcional se conoce aunque la conexión use una cuenta de servicio; el hash y la sal nunca se escriben al log	A.8.11, A.8.15, RNF-03
D-10	**Log inmutable** (trigger que aborta `UPDATE` / `DELETE` sobre `Auditoria`)	Permitir correcciones manuales del log	Un log que se puede editar no sirve como evidencia ante una auditoría; el trigger incluye instrucción controlada para deshabilitarlo en mantenimiento	RN-06, A.8.15

#	Decisión	Alternativa descartada	Justificación (por qué)	Respaldo
D-11	**Concurrencia optimista** con `RowVersion`	Bloqueo pesimista de filas o "el último que guarda gana"	Evita que dos usuarios se sobrescriban silenciosamente y no bloquea la tabla durante la edición (mejor para 2–5 usuarios concurrentes)	RF-22
D-12	**Movimientos de stock como única vía de cambio de existencias** (SP transaccional con `UPDLOCK` y `THROW`)	Editar el campo `Stock` directamente en el mantenedor	Garantiza que toda variación de inventario tenga un movimiento con motivo y documento de respaldo, y que el stock nunca quede negativo	RN-03, RF-12
D-13	**Vista `vw_ProductoDetalle`** con los cálculos de criticidad, vencimiento y reposición sugerida	Calcular en C# o repetir los `JOIN` en cada SP	Un único lugar para la regla de negocio del stock crítico; los SPs consumen la vista y la aplicación no escribe SQL de negocio	RF-11, RNF-12
D-14	**QuestPDF** (licencia comunitaria) para los reportes	iText7 (AGPL y licencia comercial), PdfSharp (API más laboriosa), Crystal Reports (dependencia pesada)	Es multiplataforma (permite **generar y validar los PDF en macOS**), es código abierto con licencia comunitaria gratuita y su API fluida reduce el código de maquetación	RF-18, RF-19
D-15	**Formularios WinForms construidos en código**	Diseñador gráfico de Visual Studio	VS Code en macOS no tiene diseñador; escribirlos en código permite compilar (y validar) la UI en macOS con `EnableWindowsTargeting` y que el port a Windows no requiera rediseñar, con el diseño centralizado y versionable	Criterio 6, portabilidad
D-16	**Cadena de conexión externalizada** (`SIGBIM_CONN` → `appsettings.json` → valor por omisión)	Cadena escrita en el código fuente	Evita versionar credenciales, permite mover la aplicación entre equipos sin recompilar y cumple con la gestión de configuraciones	A.8.9, A.8.12, RNF-02
D-17	**`PRO205.SmokeTest` (consola) como herramienta de verificación**	Probar solo con clics en la interfaz	Permite verificar DAL, BLL y reportes de forma repetible y en macOS (donde la UI no se ejecuta), dejando evidencia objetiva del funcionamiento	Criterio 7 de la rúbrica
D-18	**Retención del log de 12 meses y consulta acotada** (rango máximo de 12 meses, límite de registros)	Log sin límites	Evita degradación del rendimiento en consultas históricas y define una política de retención alineada a la supervisión periódica	RF-17, A.8.16, RNF-04

4. Cómo responde el proyecto a cada criterio de la rúbrica

Criterio	Nivel esperado (Excelente)	Evidencia en el repositorio
1. Control de versiones en GitHub	Proyecto 100 % versionado, historial incremental y descriptivo	24 commits temáticos del proyecto (`docs` → `database` → `src` por capa → `tools` → `scripts` / `docs` → correcciones de una auditoría técnica interna → limpieza), estructura `/docs`, `/database`, `/src`, `/scripts`
2. Informe con tríada e ISO 27001	Problemática, RF/RNF con ISO 27001 y tríada en todos los requisitos	`docs/01` (problemática real) · `docs/02` (25 RF con Rol→Necesidad→Criterio, 12 RNF y matriz de 15 controles del Anexo A)
3. Modelado UML	Casos de uso (CRUD) y clases alineados con el informe	`docs/03` (15 CU, include/extend justificados, matriz actor-CU) · `docs/04` (12 clases con atributos, visibilidad, métodos, multiplicidad y mapa a tablas)
4. Modelo de BD, normalización y SPs	Relacional normalizado con integridad referencial y SPs completos	`docs/05` (1FN→3FN partiendo de la planilla real) · `database/01` a `03` (10 tablas, 37 SPs, 7 triggers, 8 índices)
5. Diseño y estructura en capas	4 capas con bajo acoplamiento y documentación	`docs/06` (responsabilidades, regla de dependencia, 4 diagramas de secuencia) · 6 proyectos en `src/` con referencias unidireccionales
6. Mantenedor CRUD funcional llamando a los SPs	CRUD completo a través de las capas y de los SPs	`src/PRO205.UI.WinForms` (13 archivos) → `PRO205.BLL` (10 servicios) → `PRO205.DAL` (10 repositorios) → SPs
7. Validaciones y manejo de errores SQL	Validación en UI y BLL + captura de errores de base de datos	Validación de formato en UI, reglas y permisos en BLL, traducción de `SqlException` en `RepositorioBase`, `THROW` con códigos 51001–51050 en los SPs
Extra: reportes PDF	Módulo exigido en el Hito 2	`src/PRO205.Reportes` + `docs/evidencias/*.pdf` (reportes generados y verificados)
Extra: documentación en PDF	La entrega del Hito 1 se evalúa en PDF	`docs/pdf/` (6 documentos con portada, índice y diagramas renderizados + informe consolidado), regenerable con `bash scripts/generar_documentos_pdf.sh`

5. Guion de demostración (secuencia sugerida para la entrega)

1. **README** en GitHub: mostrar la estructura `/docs`, `/database`, `/src`, `/scripts` y el historial de commits.
2. **Informe y requisitos**: abrir `docs/01` y `docs/02` (o su versión PDF en `docs/pdf/`) y mostrar un RF completo con su tríada y su control ISO.
3. **UML**: abrir `docs/03` y `docs/04` (Mermaid renderizado en GitHub y en el PDF) y la normalización de `docs/05`.
4. **Base de datos en Windows**: ejecutar los 4 scripts con `sqlcmd` y mostrar el resultado (37 SPs, 7 triggers, semilla).
5. **Aplicación**: `dotnet run --project PRO205.UI.WinForms`, iniciar sesión como `admin` y cambiar la contraseña.
6. **CRUD del insumo**: crear un insumo, modificar su precio, filtrar por stock crítico y dar de baja un registro.
7. **Movimientos**: registrar una entrada y una salida; luego intentar una salida mayor al stock (rechazo de la regla).
8. **Trazabilidad**: abrir el log, mostrar el detalle JSON de un `UPDATE` y generar el **reporte PDF**.
9. **Seguridad**: autenticarse como `auditor`, comprobar módulos deshabilitados y mostrar el evento `ACCESO_DENEGADO`.

10. **Verificación automatizada**: ejecutar `scripts\verificar_entorno.ps1` y `dotnet run --project tools\PRO205.SmokeTest`.

---

6. Estado de la entrega y pendientes por entorno

Entregable	Estado	Observación
Documentación Hito 1 (`docs/01` a `docs/06`)	✅ Completa	Versionada en el commit `docs: informe tecnico, requisitos RF/RNF con ISO 27001, UML y arquitectura en 4 capas`
Documentación Hito 1 en **PDF** (`docs/pdf/`)	✅ Completa	6 documentos + informe consolidado (31 páginas) con **portada institucional (logo AIEP)**, índice, diagramas renderizados, encabezado y paginación
Base de datos (`database/01` a `05`)	✅ Completa	Ejecutar en Windows (SQL Server) para la validación final
Código C# en 4 capas + reportes + UI	✅ Completa y compilando	`dotnet build src/PRO205.Mantenedor.sln` → 0 errores, 0 advertencias (macOS)
Pruebas y evidencias	🟡 Parcial	Evidencia E-06 (PDF) lista; E-01 a E-05, E-07 y E-08 se generan en Windows
Scripts de operación (`scripts/`)	✅ Completos	Respaldo, verificación en Windows (`.ps1`) y en macOS (`.sh`)
Manual Windows y guía paso a paso	✅ Completos	`docs/08` y `docs/09`
Repositorio publicado en GitHub	✅ Completo	Remoto `github` creado y sincronizado (`git ls-remote github` coincide con el `HEAD` local)
Auditoría técnica independiente	✅ Completa	`docs/auditoria/Analisis_Tecnico_y_Plan_de_Mejoras.pdf` — revisión de código, BD, documentación y control de versiones, con hallazgos y correcciones aplicadas

Pendientes que dependen del equipo Windows de entrega

1. Ejecutar los scripts de `database/` y verificar 37 SPs, 7 triggers y 12 insumos cargados.
2. Crear `appsettings.json` con el servidor SQL Server local.
3. Ejecutar las pruebas CP-01 a CP-30 y capturar las evidencias E-01 a E-05, E-07 y E-08.
4. Ejecutar `scripts/verificar_entorno.ps1` y el respaldo de la base.

Pendiente de decisión (no depende del entorno Windows)

5. **Fijar GitHub como remoto principal (`origin`).** El repositorio ya está publicado y sincronizado en GitHub (remoto `github`), pero `origin` —el remoto que usa `git push` cuando no se indica ninguno— todavía apunta a Azure DevOps. Mientras eso no se decida, un `git push` sin argumentos actualiza el repositorio que la rúbrica no evalúa. Este es un cambio de configuración local de Git y se deja como acción explícita del estudiante:

   ```
   git remote set-url origin https://github.com/<usuario>/<repositorio>.git
   ```

   > **Seguridad (ISO 27001 A.5.17 y A.8.24).** El remoto de Azure DevOps tenía un *token de acceso personal* embebido en la URL (`https://usuario:TOKEN@dev.azure.com/...`), visible con `git remote -v`. Ese token se **eliminó del archivo `.git/config`** y debe **revocarse y regenerarse** en Azure DevOps (Personal Access Tokens), ya que quedó expuesto en el equipo. Para autenticar sin volver a escribir credenciales en el repositorio utilice el llavero del sistema operativo (`git config --global credential.helper osxkeychain` en macOS o *Git Credential Manager* en Windows).

> Los PDF del Hito 1 ya están generados y versionados en `docs/pdf/`, por lo que **no requieren regenerarse en Windows**. Solo si se edita algún Markdown de `/docs` hace falta volver a ejecutar el generador en un equipo con Node.js y Chrome/Edge instalados (`bash scripts/generar_documentos_pdf.sh`).

---

7. Conclusión

El SIGBIM demuestra cómo un problema real de bodega se resuelve con un diseño disciplinado: primero se delimita el problema y se especifican requisitos verificables con controles de seguridad explícitos, después se modela el dato (normalizado) y la interacción, y solo entonces se programa sobre una arquitectura en capas. La decisión central — **llevar la seguridad y la trazabilidad al motor de base de datos** — es la que hace que el sistema sea auditable: el control de acceso, la integridad y el registro de cada cambio existen aunque la aplicación cambie, y la evidencia queda disponible en un reporte PDF que el propio sistema registra como evento auditado.

PARTE 10 DE 10

Versión Final Multiplataforma (macOS)

Producto mínimo viable ejecutable: menú reorganizado, bitácora en PDF, pruebas y evidencias

Documento 10 del repositorio (docs/10_Version_Final_macOS.md)

Ficha del documento	
Proyecto	SIGBIM — Sistema de Gestión de Bodega de Insumos Médicos
Asignatura	Desarrollo e Ingeniería de Software (PRO205)
Institución	AIEP — Universidad Andrés Bello
Estudiante	Jonathan Felipe Urbina Monasterio — `JONATHAN.URBINA@correoaiep.cl`
Documento	`docs/10_Version_Final_macOS.md`
Fecha de emisión	octubre de 2026

Este documento describe la **versión final ejecutable** del mantenedor SIGBIM, construida y probada en macOS. Mantiene el mismo diseño de los documentos 01 a 09 (cuatro capas, procedimientos almacenados, integridad referencial y controles ISO/IEC 27001), pero con un alcance reducido a un **producto mínimo viable**, y agrega tres mejoras de uso: consulta de productos de solo lectura, un punto único para administrar el catálogo y la descarga de la bitácora en PDF.

---

1. Por qué existe una versión para macOS

Aspecto	Proyecto base (documentos 01 a 09)	Versión final (este documento)
Interfaz	Windows Forms	**Avalonia UI** (mismo modelo de ventanas, controles y eventos)
Base de datos	SQL Server	**MySQL 8**
Sistema operativo	Windows	**macOS** (también Windows y Linux)
Capas BLL, DAL y Entidades	C# / .NET	C# / .NET 8 (misma estructura)

Windows Forms depende de la API Win32 y **no se ejecuta en macOS**. Como el equipo de desarrollo es un Mac, la capa de presentación se implementó con Avalonia UI. Esto solo cambia la capa UI: las capas de negocio, acceso a datos y entidades siguen el mismo diseño, lo que también demuestra el bajo acoplamiento de la arquitectura (criterio 5 de la rúbrica).

2. Alcance del producto mínimo viable

La versión final se concentra en el núcleo del inventario: **4 tablas** con integridad referencial completa.

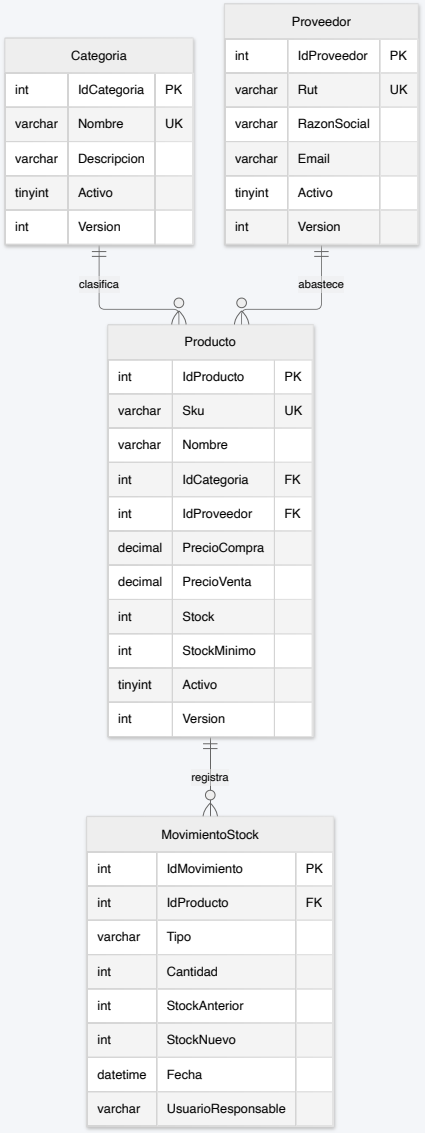


Figura 1 — Diagrama del documento (fuente Mermaid versionada en el repositorio).

- Todas las claves foráneas usan `ON DELETE RESTRICT`: no se puede borrar una categoría o un proveedor que tenga productos. Por eso la aplicación usa **baja lógica** (`Activo = 0`, ISO A.5.33).
- Cada tabla maestra tiene columnas de auditoría (`FechaCreacion`, `UsuarioCreacion`, `FechaModificacion`, `UsuarioModificacion`) y un campo `Version` para el control de concurrencia optimista.
- La vista `vw_productodetalle` une Producto, Categoría y Proveedor y calcula margen, estado de stock y valor del inventario, sin duplicar datos (3FN).
- `CHECK` en la base de datos: precios ≥ 0, stock ≥ 0, cantidad > 0 y tipo ∈ {ENTRADA, SALIDA, AJUSTE}.

Fuera del alcance (mejoras futuras): las tablas `Usuario`, `Rol`, `Permiso`, `RolPermiso`, `UnidadMedida` y `Auditoria` del modelo completo. El diseño permite agregarlas sin modificar las 4 tablas actuales (ver sección 9).

3. Arquitectura y menú de la versión final

La solución `src/Sigbim.sln` tiene los cuatro proyectos de la arquitectura: `Sigbim.Entidades`, `Sigbim.DAL` (MySqlConnector, solo `CALL sp_…` con parámetros), `Sigbim.BLL` (reglas de negocio, validaciones y bitácora) y `Sigbim.UI` (Avalonia + QuestPDF). La UI **no** referencia la DAL.

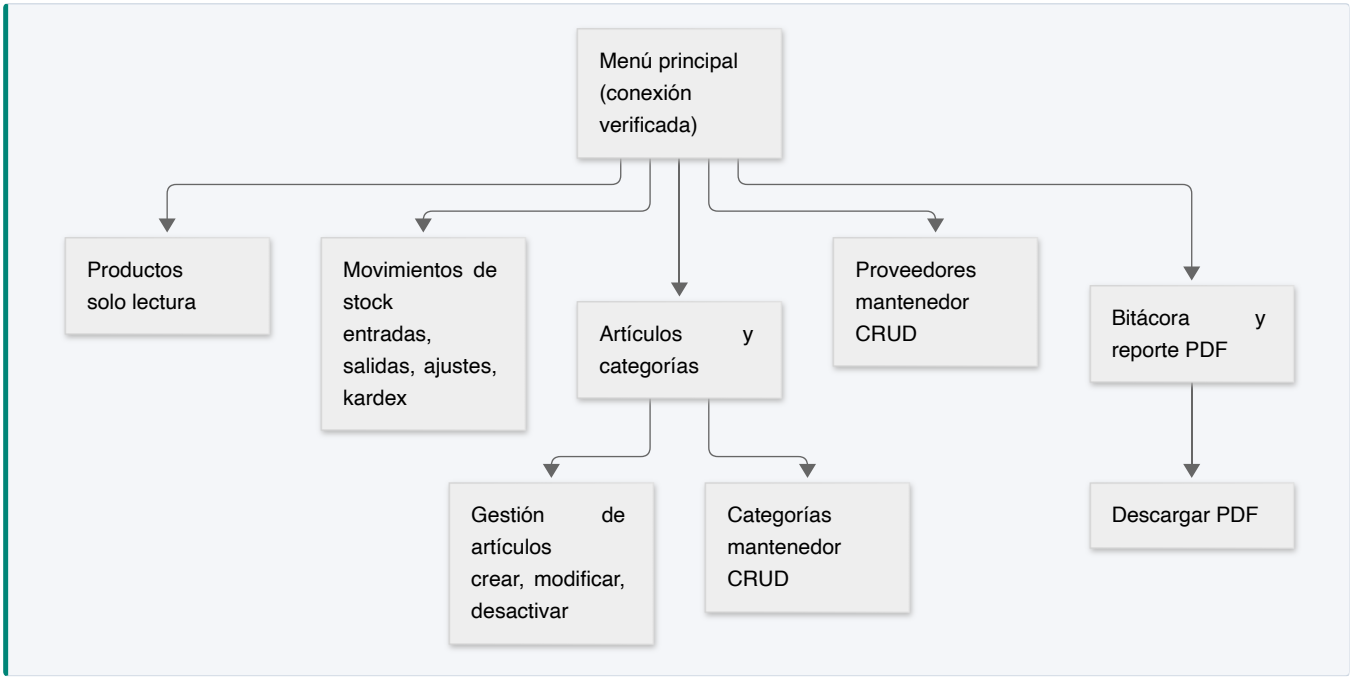


Figura 2 — Diagrama del documento (fuente Mermaid versionada en el repositorio).

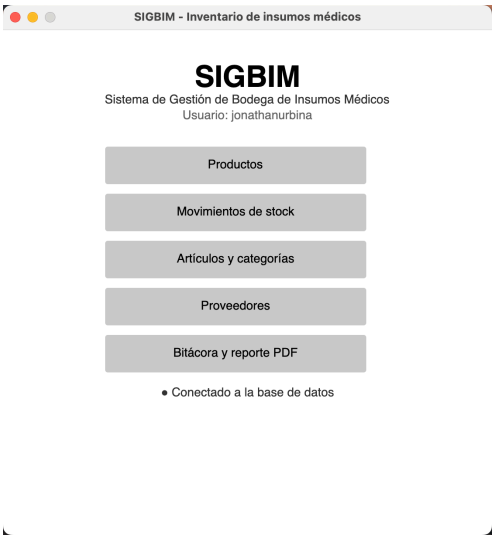


Figura — Menú principal de la versión final, conectado a la base de datos.

4. Funcionalidades agregadas en la versión final

4.1 Productos: consulta de solo lectura

El menú **Productos** muestra todos los productos registrados con buscador por SKU o nombre, categoría, proveedor, precios, stock, estado de stock y vencimiento. **No tiene formulario ni botones de edición**: seleccionar una fila no la abre para modificarla. Se implementó con un parámetro `soloLectura` en la plantilla genérica `VentanaMantenedor<T>`, de modo que la misma clase `VentanaProductos` sirve para la consulta y para la gestión, sin duplicar código.

4.2 Artículos y categorías: un único punto de administración

Todas las operaciones que **modifican el catálogo** quedaron en un solo apartado:

- **Gestión de artículos:** crear, modificar y desactivar productos (RF-01, RF-03, RF-04).
- **Categorías:** crear, listar, modificar y desactivar categorías (RF-05 a RF-08).

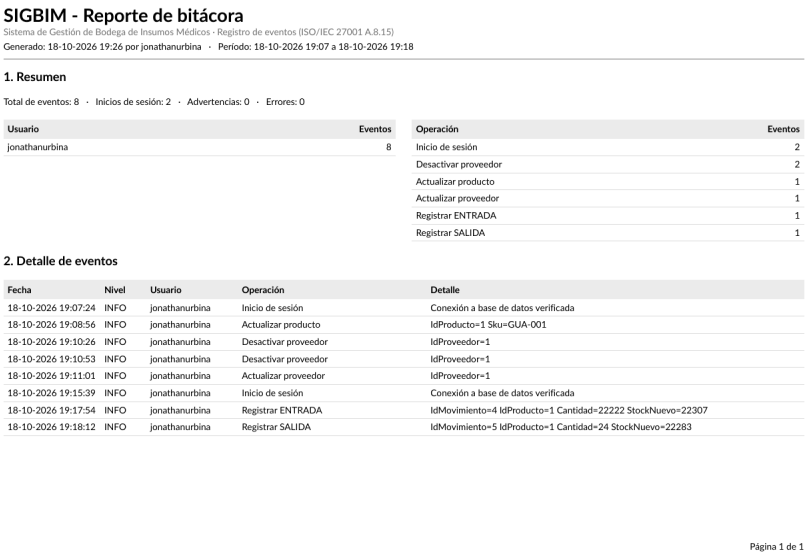
Separar la consulta de la gestión reduce las modificaciones accidentales y deja claro dónde se administra la información.

4.3 Bitácora y reporte PDF

ID	Rol / Usuario	Necesidad / Acción	Criterio de aceptación	Criterio de seguridad (ISO 27001)
RF-17	Administrador / Auditor	Consultar la bitácora: inicios de sesión, creaciones, modificaciones, desactivaciones y movimientos.	La ventana lista todos los eventos (más reciente primero) con fecha, nivel, usuario, operación y detalle.	A.8.15 / A.8.16: revisión de la actividad. A.8.12: sin contraseñas ni cadena de conexión.

ID	Rol / Usuario	Necesidad / Acción	Criterio de aceptación	Criterio de seguridad (ISO 27001)
RF-18	Administrador / Auditor	Descargar la bitácora completa en PDF.	El PDF incluye quién lo generó, cuándo, el período cubierto, un resumen por usuario y por operación y el detalle cronológico. Se abre automáticamente.	A.8.15: cada exportación queda registrada (`Exportar bitácora PDF`), así que también es trazable quién extrajo la información.

La lectura de la bitácora está en la BLL (`RegistroEventos.Leer()` y `BitacoraBLL`), la entidad `EventoBitacora` en la capa de Entidades y la generación del PDF (`ReporteBitacoraPdf`, con QuestPDF) en la UI, porque el PDF es un formato de presentación.



SIGBIM - Reporte de bitácora

Sistema de Gestión de Bodega de Insumos Médicos · Registro de eventos (ISO/IEC 27001 A.8.15)

Generado: 18-10-2026 19:26 por jonathanurbina · Período: 18-10-2026 19:07 a 18-10-2026 19:18

1. Resumen

Total de eventos: 8 · Inicios de sesión: 2 · Advertencias: 0 · Errores: 0

Usuario	Eventos
jonathanurbina	8

Operación	Eventos
Inicio de sesión	2
Desactivar proveedor	2
Actualizar producto	1
Actualizar proveedor	1
Registrar ENTRADA	1
Registrar SALIDA	1

2. Detalle de eventos

Fecha	Nivel	Usuario	Operación	Detalle
18-10-2026 19:07:24	INFO	jonathanurbina	Inicio de sesión	Conexión a base de datos verificada
18-10-2026 19:08:56	INFO	jonathanurbina	Actualizar producto	IdProducto=1 Sku=GUA-001
18-10-2026 19:10:26	INFO	jonathanurbina	Desactivar proveedor	IdProveedor=1
18-10-2026 19:10:53	INFO	jonathanurbina	Desactivar proveedor	IdProveedor=1
18-10-2026 19:11:01	INFO	jonathanurbina	Actualizar proveedor	IdProveedor=1
18-10-2026 19:15:39	INFO	jonathanurbina	Inicio de sesión	Conexión a base de datos verificada
18-10-2026 19:17:54	INFO	jonathanurbina	Registrar ENTRADA	IdMovimiento=4 IdProducto=1 Cantidad=22222 StockNuevo=22307
18-10-2026 19:18:12	INFO	jonathanurbina	Registrar SALIDA	IdMovimiento=5 IdProducto=1 Cantidad=24 StockNuevo=22283

Página 1 de 1

Figura — Primera página del reporte PDF de la bitácora generado por la aplicación.

5. Seguridad aplicada (ISO/IEC 27001:2022)

Control	Aplicación en la versión final	Evidencia
A.8.2 / A.8.3 Mínimo privilegio	La aplicación usa `sigbim_app`, que solo tiene `EXECUTE` sobre los SPs.	`SELECT` y `UPDATE` directos devuelven *ERROR 1142 … command denied*.
A.8.28 Codificación segura	Solo SPs con parámetros, sin SQL concatenado.	`Db.cs`: `CALL sp_…(@p_…)`.
A.5.17 / A.8.9 Credenciales	La cadena de conexión está en `appsettings.json` (excluido de Git) o en la variable `SIGBIM_CONEXION`.	`.gitignore` y `appsettings.example.json`.
A.8.15 Registro	Columnas de auditoría, kardex inmutable y bitácora `logs/` con descarga en PDF.	Bitácora y reporte PDF (sección 4.3).
A.8.12 Fuga de datos	El usuario ve mensajes comprensibles; el detalle técnico queda en la bitácora.	`TraductorErrores` y `Ui.EjecutarAsync`.
A.5.33 Protección de registros	Baja lógica en productos, categorías y proveedores.	`sp_*_eliminar` hace `Activo = 0`.
A.8.32 Gestión de cambios	Cada cambio tiene su propio commit descriptivo.	`git log --oneline`.

6. Pruebas y evidencias

Se ejecutaron **25 casos de prueba**, todos aprobados (detalle completo en `docs/07_pruebas_y_evidencias.md` de la versión final).

Nivel	Ejemplos	Casos	Aprobados
Instalación y compilación	Scripts SQL, `dotnet build` (0 errores, 0 advertencias), conexión al iniciar.	4	4
Mínimo privilegio	`CALL` permitido; `SELECT` y `UPDATE` directos rechazados.	3	3
Reglas de la BD	Stock insuficiente, cantidad 0, tipo inválido, producto inexistente, precio de venta menor al de compra, SKU y categoría duplicados.	7	7
Validaciones BLL	RUT módulo 11 y email inválidos, campos obligatorios, precios, no desactivar productos con stock.	5	5
Funcionales	Modificar producto, desactivar proveedor, entrada y salida de stock, exportar la bitácora, menú reorganizado.	6	6
Total		**25**	**25**

En las pruebas negativas de la base de datos se comprobó que **el stock no cambió** (GUA-001: 22.283 y PAR-500: 200, antes y después), lo que confirma que los SPs revierten la operación (ROLLBACK) ante un error.

Incidente corregido durante la instalación

La aplicación recibía `ERROR 1045: Access denied for user 'sigbim_app'@'localhost'` aunque el usuario existía. La causa era una cuenta antigua `sigbim_app@localhost`, que MySQL prefería sobre la nueva `sigbim_app@%` por ser más específica. Se corrigió el script `02_seguridad_usuario_app.sql` para eliminar ambas cuentas antes de crear la nueva, y la corrección quedó en un commit propio.

7. Ejecución en macOS

```
# 1. Base de datos (pide la clave de root)
cd database
mysql -u root -p < 01_sigbim_inventario.sql
mysql -u root -p < 02_seguridad_usuario_app.sql

# 2. Configuración de la conexión
cd ../src/Sigbim.UI
cp appsettings.example.json appsettings.json   # ajustar la clave de sigbim_app si se cambió

# 3. Ejecutar
cd ..
dotnet run --project Sigbim.UI
```

Requisitos: .NET SDK 8 o superior y MySQL Server 8. La primera ejecución descarga los paquetes Avalonia, MySqlConnector y QuestPDF.

8. Correspondencia con la rúbrica

Criterio	Cómo lo cumple la versión final
1. Control de versiones	Repositorio con historial incremental: documentación, base de datos, cada capa y cada ajuste final en commits separados y descriptivos.
2. Informe, tríada e ISO 27001	Documentos 01 a 07 de la versión final; RF con tríada (incluidos RF-17 y RF-18) y matriz de controles.
3. UML	Casos de uso (CU-01 a CU-16) y diagrama de clases por capa actualizados con el menú y la bitácora.
4. BD, normalización y SPs	4 tablas en 3FN con PK, FK, UNIQUE y CHECK; vista consolidada; CRUD completo con SPs.
5. Arquitectura en 4 capas	UI → BLL → DAL → BD, con Entidades compartidas; la UI no referencia la DAL.
6. Mantenedor CRUD	Crear, consultar, modificar y desactivar productos, categorías y proveedores, más movimientos de stock, todo mediante SPs.
7. Validaciones y errores	Validación en UI (formato), BLL (reglas) y BD (restricciones y SPs); errores SQL traducidos a mensajes de usuario.

9. Mejoras futuras

1. **Inicio de sesión con usuarios y roles** (`Usuario`, `Rol`, `Permiso`, `RolPermiso`): las columnas de auditoría pasarían de texto a claves foráneas hacia `Usuario`.
2. **Unidad de medida**: `CREATE TABLE UnidadMedida` + `ALTER TABLE Producto ADD IdUnidadMedida`.
3. **Auditoría en base de datos**: tabla `Auditoria` con triggers, como complemento de la bitácora de archivos.
4. **Filtros del reporte PDF** por fecha, usuario u operación.
5. **Botón "Volver al inicio"** en cada ventana (hoy se vuelve cerrando la ventana).